A PROJECT REPORT ON

ClimateSure

A Parametric Climate Insurance Platform built with Spring Boot Microservices and Angular

SUBMITTED BY

Akshaya Chigullapally

Full-Stack Development Project

Java 17 | Spring Boot 4.1 | Spring WebFlux & R2DBC | Spring Cloud Gateway & Eureka | Angular 21 | NgRx | Tailwind CSS | JWT & RBAC

October 2026

Abstract

Traditional crop and property insurance pays only after a loss adjuster has inspected the damage. For small farmers, households and small businesses exposed to extreme weather this means weeks of paperwork, disputes about the size of the loss and money that arrives long after it was needed. **Parametric insurance** takes a different approach: the policy names a measurable index, such as rainfall in a zone over 24 hours, and an agreed threshold. When an independent measurement crosses the threshold, the claim is created and paid automatically without assessing the individual loss.

This report presents **ClimateSure**, a full-stack parametric climate insurance platform. The backend is a set of six reactive **Spring Boot 4.1** microservices (Auth, Product & Policy, Claims, Trigger, Financial and Notification), discovered through a **Eureka** registry and exposed through a single **Spring Cloud Gateway**. Each service owns its own H2 database accessed through non-blocking **R2DBC**. The frontend is an **Angular 21** single-page application using **NgRx** for state management, Tailwind CSS for styling and Server-Sent Events for live notifications.

The platform supports four roles: customer, underwriter, claims officer and administrator. A customer can obtain a quote, apply for a policy, pay the premium and follow claims and payouts. An underwriter approves applications against zone exposure limits and weather alerts. An administrator configures products, trigger thresholds, zones, perils and pricing rules and simulates weather readings. When a reading crosses a product threshold, the **trigger engine** verifies the event, finds every active and paid policy in the zone and creates the claims. After a claims officer approves a claim, the payout is made, the claim is settled and the customer is notified in real time. Security uses **JWT** access and refresh tokens, **Spring Security** filters in every service and **role-based access control** enforced at both the gateway and the service level.

The system exposes 124 REST endpoints through the gateway and 19 internal service-to-service endpoints across 28 database tables. It is verified by 577 backend tests (JUnit 5, Mockito, Reactor StepVerifier) and 293 frontend tests (Vitest), all passing. The report describes the architecture, database design, UML models, user journeys, functional requirements, API, security model, validation strategy, testing, logging and monitoring. It ends with screenshots of the working system, its limitations and planned enhancements.

Keywords: parametric insurance, microservices, Spring WebFlux, R2DBC, API gateway, Eureka, JWT, RBAC, Angular, NgRx, Server-Sent Events.

Table of Contents

Abstract 2

1. Introduction 9

1.1 Background 9
1.2 Problem Statement 9
1.3 Objectives 9
1.4 Scope 10
1.5 Organisation of the Report 10

2. System Architecture & Microservices 11

2.1 Architectural Style 11
2.2 High-Level Architecture 11
2.3 Microservice Catalogue 12
2.4 Inter-Service Communication 12
2.5 Request Life-Cycle through the Gateway 14
2.6 Scheduled Background Jobs 15
2.7 Frontend Architecture 15

3. ER Diagram & Database Design 17

3.1 Design Approach 17
3.2 Auth Service Schema 18
3.3 Product & Policy Service Schema 19
3.4 Claims Service Schema 20
3.5 Trigger Service Schema 21
3.6 Financial and Notification Service Schemas 22
3.7 Cross-Service References 23
3.8 Data Integrity Measures 23

4. UML Diagrams 24

4.1 Use Case Diagram 24
4.2 Class Diagrams 24
4.3 Sequence Diagrams 27
4.4 State Diagrams 28

5. User Journey & Flow 30

5.1 User Roles 30
5.2 Customer Journey 31
5.3 Staff Journeys 32
5.4 Journey Catalogue 32
5.5 Frontend Page Map 33

6. Functional Requirements & Business Rules 34

6.1 Functional Requirements 34
6.2 Non-Functional Requirements 35
6.3 Business Rules 35
6.4 Premium Calculation 36

7. API Documentation 37
7.1 Conventions 37
7.2 Endpoint Summary 37
7.3 Endpoint Reference 37
7.4 Internal Endpoints 44
7.5 Sample Requests and Responses 44

8. Security & Role-Based Access Control 46
8.1 Security Overview 46
8.2 JSON Web Tokens 46
8.3 Spring Security Configuration 47
8.4 Role-Based Access Matrix 48
8.5 Additional Security Measures 48

9. Validation, DTOs & Exception Handling 49
9.1 DTO Design 49
9.2 Backend Validation 49
9.3 Frontend Validation 50
9.4 Exception Handling 50

10. Technology Stack & Project Structure 52
10.1 Technology Stack 52
10.2 Backend Project Structure 53
10.3 Frontend Project Structure 53
10.4 Deployment View 54

11. Testing & Code Coverage 55
11.1 Testing Strategy 55
11.2 Backend Results 55
11.3 Frontend Results 56
11.4 Sample Test 57

12. Logging & Monitoring 58
12.1 Logging 58
12.2 Monitoring 59

13. Results 60
13.1 Public Pages 60
13.2 Customer Workspace (akshaya@gmail.com) 62
13.3 Underwriter Workspace 66
13.4 Claims Officer Workspace 67

13.5 Admin Workspace 68
13.6 Summary of Results 71

14. Assumptions, Limitations & Future Enhancements 72
14.1 Assumptions 72
14.2 Limitations 72
14.3 Future Enhancements 72

15. Conclusion 74

References 75

Appendix A — Running the Project 76

List of Figures

Figure 2.1: High-level system architecture of ClimateSure 11
Figure 2.2: Request processing through the gateway and a service 14
Figure 2.3: Frontend architecture of the Angular application 16
Figure 3.1: ER diagram — Auth Service (authdb) 18
Figure 3.2: ER diagram — Product & Policy Service (productdb) 19
Figure 3.3: ER diagram — Claims Service (claimsdb) 20
Figure 3.4: ER diagram — Trigger Service (triggerdb) 21
Figure 3.5: ER diagram — Financial Service (financialdb) and Notification Service (notificationdb) 22
Figure 3.6: Logical references between service databases (by identifier, no physical foreign key) 23
Figure 4.1: Use case diagram 24
Figure 4.2: Domain class diagram 25
Figure 4.3: Layered class diagram — Claims Service 26
Figure 4.4: Sequence diagram — login, token use and refresh 27
Figure 4.5: Sequence diagram — quote, application, underwriting and premium payment 27
Figure 4.6: Sequence diagram — from weather reading to settled claim 28
Figure 4.7: Policy life-cycle 28
Figure 4.8: Claim life-cycle 28
Figure 5.1: Customer journey flow 31
Figure 5.2: Staff journeys and hand-offs 32
Figure 8.1: Token refresh flow 47
Figure 11.1: Backend line coverage by service (JaCoCo) 56
Figure 13.1: Landing page 60
Figure 13.2: Sign-in page 61
Figure 13.3: Public product catalogue with trigger rules 61
Figure 13.4: Customer dashboard — Akshaya Chigullapally 62
Figure 13.5: Apply for coverage 62
Figure 13.6: My policies — two active policies and one pending application 63
Figure 13.7: Payments and payouts — premiums paid and the ₹1,50,000 payout received 63
Figure 13.8: My claims — automatic claim settled, manual claim submitted 64
Figure 13.9: Submit a manual claim with evidence 64
Figure 13.10: Live notifications — claim settled and payout processed 65
Figure 13.11: Customer profile 65
Figure 13.12: Underwriter dashboard 66
Figure 13.13: Policy applications for review 66
Figure 13.14: Claims officer dashboard 67
Figure 13.15: Claims review queue 67
Figure 13.16: Admin dashboard 68
Figure 13.17: Product management 68

Figure 13.18: Trigger simulator — last simulated value 172 mm for Weather Protection Plan 69
Figure 13.19: Verified trigger events 69
Figure 13.20: Payout history 70
Figure 13.21: User management 70

List of Tables

Table 1.1: Organisation of the report 10
Table 2.1: Microservices, ports and responsibilities 12
Table 2.2: Key internal service-to-service calls 12
Table 2.3: Scheduled jobs 15
Table 3.1: Databases and tables per service (28 tables) 17
Table 4.1: Life-cycles of other entities 29
Table 5.1: User roles and landing pages 30
Table 5.2: User journey catalogue 32
Table 5.3: Frontend page map by role 33
Table 6.1: Functional requirements 34
Table 6.2: Non-functional requirements 35
Table 6.3: Business rules enforced in the services 35
Table 7.1: Number of endpoints per service 37
Table 7.2: Auth Service endpoints 37
Table 7.3: Product & Policy Service endpoints 38
Table 7.4: Claims Service endpoints 40
Table 7.5: Trigger Service endpoints 41
Table 7.6: Financial Service endpoints 42
Table 7.7: Notification Service endpoints 43
Table 7.8: Internal service-to-service endpoints 44
Table 8.1: Security layers 46
Table 8.2: Role-based access matrix 48
Table 9.1: Examples of DTO validation rules 50
Table 9.2: Exception to HTTP status mapping 50
Table 10.1: Technology stack 52
Table 10.2: Deployment of the components 54
Table 11.1: Test levels and tools 55
Table 11.2: Backend test results and JaCoCo coverage 55
Table 11.3: Frontend test results (Vitest, V8 coverage) 56
Table 12.1: Monitoring facilities 59
Table 13.1: Objectives versus results 71
Table 14.1: Planned enhancements 72
Table A.1: Accounts used for the results 76

CHAPTER 1

1. Introduction

1.1 Background

India and many other countries face increasingly frequent climate events: cloudbursts, floods, droughts, heat waves and cyclones. People most affected by these events are often the least protected by conventional insurance. Indemnity-based insurance requires proof of the exact loss and a field inspection. This is slow, expensive to administer and hard to scale to millions of small policies.

Parametric (index-based) insurance replaces loss assessment with an objective trigger. The insurer and the customer agree in advance on a measured parameter, a location zone, a threshold and the amount to be paid. If the measured value in that zone crosses the threshold during the policy period, the payout is due. The process is transparent, fast and almost entirely automatable.

1.2 Problem Statement

The goal of this project is to design and build a secure, scalable web platform that manages the complete life-cycle of parametric climate insurance, which includes:

- product design with machine-checkable trigger rules (measurement type, unit, threshold, comparison operator, aggregation and observation window);
- risk-based premium calculation, quoting, underwriting and premium collection;
- continuous intake of weather measurements per location zone, and automatic detection of trigger events;
- automatic creation of claims for every eligible policy, human review by a claims officer and automatic payout and settlement;
- real-time notification of every important event to the right user, and strict role-based access for customers and staff.

1.3 Objectives

- Build the backend as independent, reactive microservices with a database per service, service discovery and a single API gateway.
- Secure every request with stateless JWT authentication and role-based access control enforced at the gateway and inside every service.
- Lock the trigger rule of a policy at purchase time (a *snapshot*), so later threshold changes never change an existing contract.
- Make trigger processing safe to repeat by using idempotency keys, so the same reading never creates duplicate claims or payouts.
- Provide a responsive Angular frontend with a separate, guarded workspace for each of the four roles, and live notifications.

- Validate all input at the edge with DTOs, return uniform error responses and keep the code covered by automated tests.

1.4 Scope

The project covers the full insurance flow from product set-up to payout for two parametric products: the **Weather Protection Plan** (total rainfall ≥ 150 mm in 24 hours) and the **Flood Protection Plan** (maximum water level ≥ 5 m in 6 hours). Customers can also define a custom parametric policy. Weather data is supplied by a built-in demo feed and an administrator's *trigger simulator* instead of a live meteorological API. Payment and payout gateways are simulated. Both of these are deliberate scope decisions discussed in Chapter 14.

1.5 Organisation of the Report

Table 1.1: Organisation of the report

Chapter	Contents
2	System architecture, microservices, inter-service communication and frontend architecture
3	Database design and entity-relationship diagrams for each service
4	UML models: use case, class, sequence and state diagrams
5	User journeys and process flows for each role
6	Functional and non-functional requirements and business rules
7	REST API documentation
8	Security: JWT, Spring Security and role-based access control
9	Validation, DTOs and exception handling
10	Technology stack and project structure
11	Testing strategy and code coverage
12	Logging and monitoring
13	Results with screenshots of the running application
14–15	Assumptions, limitations, future enhancements and conclusion

CHAPTER 2

2. System Architecture & Microservices

2.1 Architectural Style

ClimateSure follows a **microservice architecture**. The domain is split along business capabilities into six services, each with its own code base, its own database and its own deployable JAR. All services are **reactive**: they are built on Spring WebFlux (Netty) and use R2DBC for non-blocking database access. A small number of threads can therefore serve many concurrent requests, which matters during a weather event when thousands of policies may need processing at once.

The browser never calls a service directly. All external traffic enters through the **API Gateway**, which validates the JWT, removes any identity headers the client tried to set and forwards the request, with trusted identity headers and a shared gateway key, to the target service. Services find each other through the **Eureka** service registry and call each other with a load-balanced WebClient on dedicated `/internal/**` paths that the gateway never exposes.

2.2 High-Level Architecture

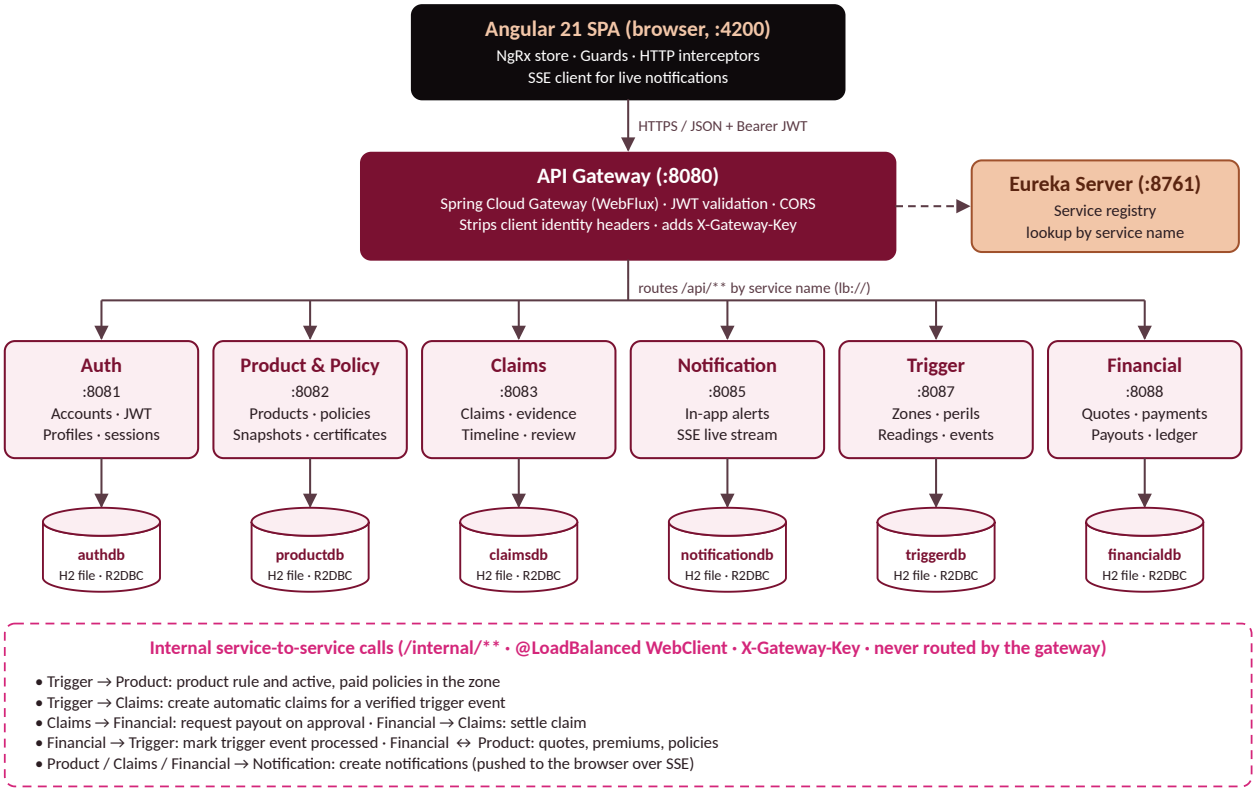


Figure 2.1: High-level system architecture of ClimateSure

2.3 Microservice Catalogue

Table 2.1: Microservices, ports and responsibilities

Service	Port	Eureka name	Responsibility	Database
Eureka Server	8761	—	Service registry. Every service registers on start-up and is looked up by name.	—
API Gateway	8080	API-GATEWAY	Single entry point. Routing, JWT validation, CORS, header sanitising, access logging, 503 fallback.	—
Auth Service	8081	AUTH-SERVICE	Registration, login, token refresh and logout, sessions, user profiles and addresses, user status (admin).	authdb (5 tables)
Product & Policy Service	8082	PRODUCT-SERVICE	Products and trigger thresholds, policy applications, underwriting, trigger snapshots, renewals, certificates (PDF), zone exposure.	productdb (6 tables)
Claims Service	8083	CLAIMS-SERVICE	Automatic and manual claims, evidence documents, review workflow, settlement, claim timeline.	claimsdb (3 tables)
Notification Service	8085	NOTIFICATION-SERVICE	In-app notifications, unread counts, live Server-Sent Events stream, clean-up job.	notificationdb (1 table)
Trigger Service	8087	TRIGGER-SERVICE	Location zones, perils, trigger definitions, measurements, trigger detection and events, idempotency.	triggerdb (7 tables)
Financial Service	8088	FINANCIAL-SERVICE	Pricing rules, quotes, premium payments, payouts, ledger, reconciliation, receipts (PDF), metrics.	financialdb (6 tables)

2.4 Inter-Service Communication

Services communicate synchronously over HTTP using Spring's reactive `WebClient` annotated with `@LoadBalanced`, so a URL such as `http://CLAIMS-SERVICE/internal/claims/from-trigger-event` is resolved through Eureka. Every internal call carries the `X-Gateway-Key` header. The receiving service accepts `/internal/**` requests only when the key matches. Table 2.2 lists the most important call paths.

Table 2.2: Key internal service-to-service calls

Caller	Target	Endpoint	Purpose
Product	Financial	POST /internal/financial/pricing/calculate	Calculate the premium for a new application
Product	Financial	POST /internal/financial/quotes/{id}/validate, PUT …/consume	Validate and consume an accepted quote
Product	Trigger	GET /internal/triggers/zones/active	Zones a customer can choose; zone details for certificates

Caller	Target	Endpoint	Purpose
Product	Trigger	PUT /internal/triggers/products/{id}/definition	Synchronise the product's trigger rule into a trigger definition
Trigger	Product	GET /internal/policies/active-for-trigger	Active, paid policies in the zone with their locked snapshot
Trigger	Claims	POST /internal/claims/from-trigger-event	Create automatic claims for a verified trigger event
Claims	Product	GET /internal/policies/{id}	Check ownership, status and period before a manual claim
Claims	Financial	POST /internal/financial/payouts	Request the payout when a claim is approved
Financial	Claims	PUT /internal/claims/{id}/settle	Settle the claim once the payout succeeds
Financial	Trigger	PUT /internal/triggers/events/{id}/processed	Mark the trigger event processed
Product, Claims, Financial	Notification	POST /internal/notifications	Create a notification (idempotency key prevents duplicates)

2.5 Request Life-Cycle through the Gateway

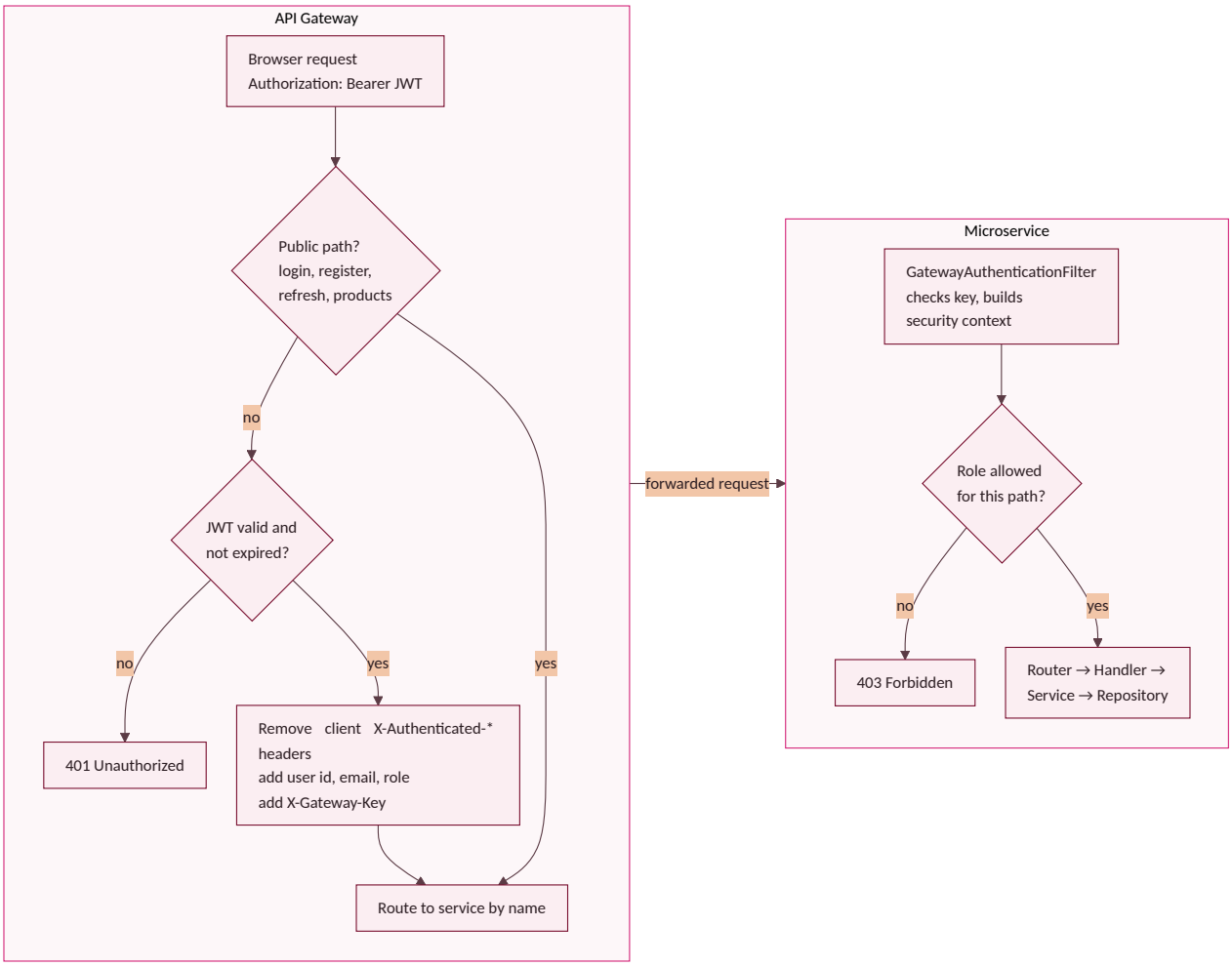


Figure 2.2: Request processing through the gateway and a service

2.6 Scheduled Background Jobs

Table 2.3: Scheduled jobs

Job	Service	Schedule	Purpose
Demo weather feed	Trigger	Every 60 s	Adds normal sample readings for every active zone (an admin-set value is held for 1 hour)
Old reading clean-up	Trigger	Hourly	Deletes demo readings older than 3 days
Policy expiry	Product	Daily 01:00	Active policies past their end date become EXPIRED
Policy renewal	Product	Daily 01:30	Auto-renew applications and renewal reminders (14 and 3 days before end)
Premium reminders	Financial	Daily 09:00	“Premium due” and “overdue” notifications
Payout sweep	Financial	Every 5 min	Pays waiting payouts and retries failed ones (maximum 3 attempts)
Reconciliation	Financial	Daily 03:30	Checks that every payment and payout has a ledger entry
Default price rules	Financial	Every 60 s	Creates a default pricing rule for any new product
Notification clean-up	Notification	Daily 03:00	Removes read notifications older than 90 days

2.7 Frontend Architecture

The Angular application is organised in layers. **Components** are standalone and use signals for local state. **NgRx** holds the shared state per feature (auth, policies, claims, payments, payouts, notifications, products, quotes, zones, perils, pricing rules, trigger events and users). Each feature has actions, a reducer, selectors and effects. Effects call the **feature services**, which wrap Angular's `HttpClient`. All HTTP traffic passes through two functional **interceptors**: `authInterceptor` attaches the bearer token and refreshes it transparently, and `errorInterceptor` maps errors to user-friendly toast messages. Route **guards** (`authGuard`, `guestGuard`, `roleGuard`) protect every workspace.

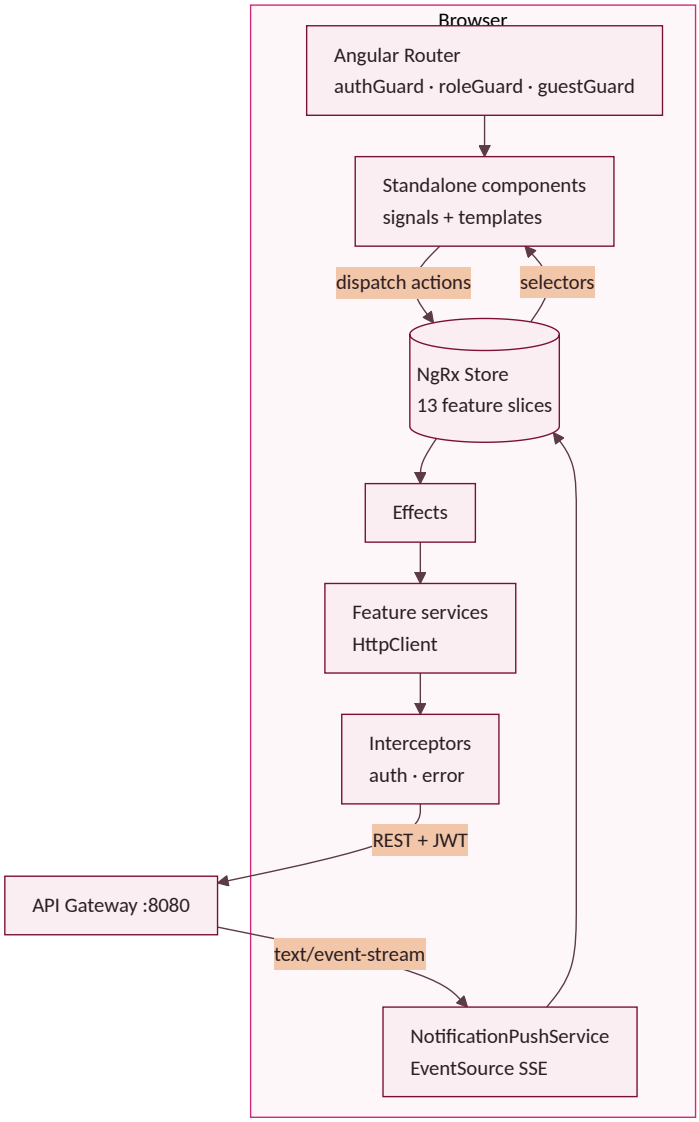


Figure 2.3: Frontend architecture of the Angular application

CHAPTER 3

3. ER Diagram & Database Design

3.1 Design Approach

The system follows the **database-per-service** pattern. Each microservice owns a private H2 database file (for example `./data/claimsdb`), accessed through R2DBC, and creates its schema at start-up from `schema.sql`. No service reads another service's tables. Inside a service, relationships are enforced with foreign keys. Across services, a row stores only the *identifier* of the remote entity (for example `claims.policy_id`), and the owning service is asked through an internal API when details are needed. Frequently displayed values such as policy number and product name are copied (denormalised) so lists can be shown without extra calls.

Monetary values use `DECIMAL(15,2)` or `DECIMAL(19,2)` and Java `BigDecimal`. Measurement values use `DECIMAL(19,4)`. Business references (`POL-2026-…`, `CLM-2026-…`, `PAY-…`) are unique columns separate from the numeric primary keys.

Table 3.1: Databases and tables per service (28 tables)

Database	Tables
authdb	users_credentials, user_profiles, addresses, refresh_tokens, sessions
productdb	products, coverages, policies, policy_trigger_snapshots, policy_status_history, product_threshold_history
claimsdb	claims, claim_timeline, claim_documents
triggerdb	location_zones, perils, trigger_definitions, observations, current_measurements, trigger_events, idempotency_records
financialdb	pricing_rules, quotes, payments, payouts, financial_transactions, reconciliation_records
notificationdb	notifications

3.2 Auth Service Schema

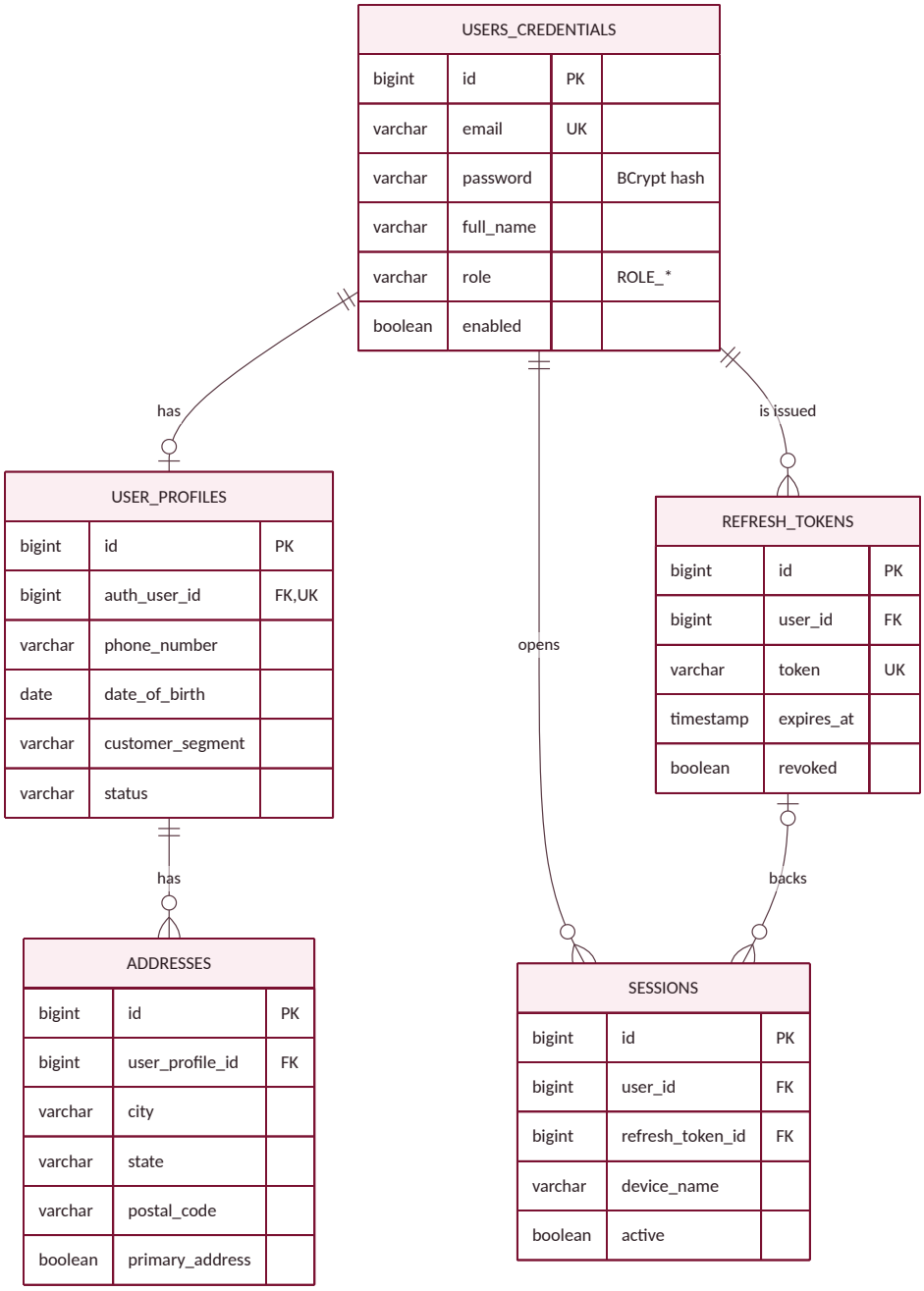


Figure 3.1: ER diagram — Auth Service (authdb)

3.3 Product & Policy Service Schema

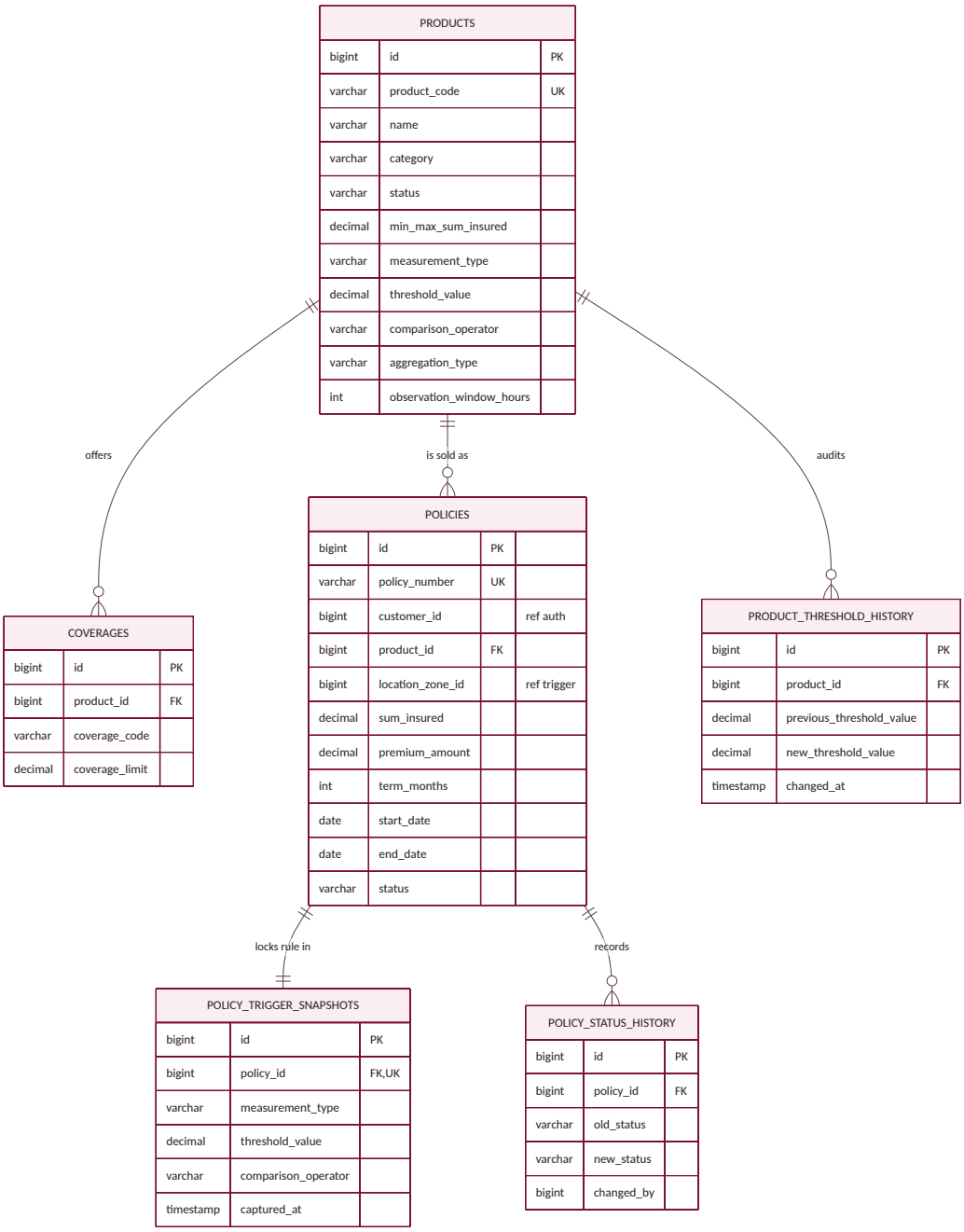


Figure 3.2: ER diagram — Product & Policy Service (productdb)

3.4 Claims Service Schema

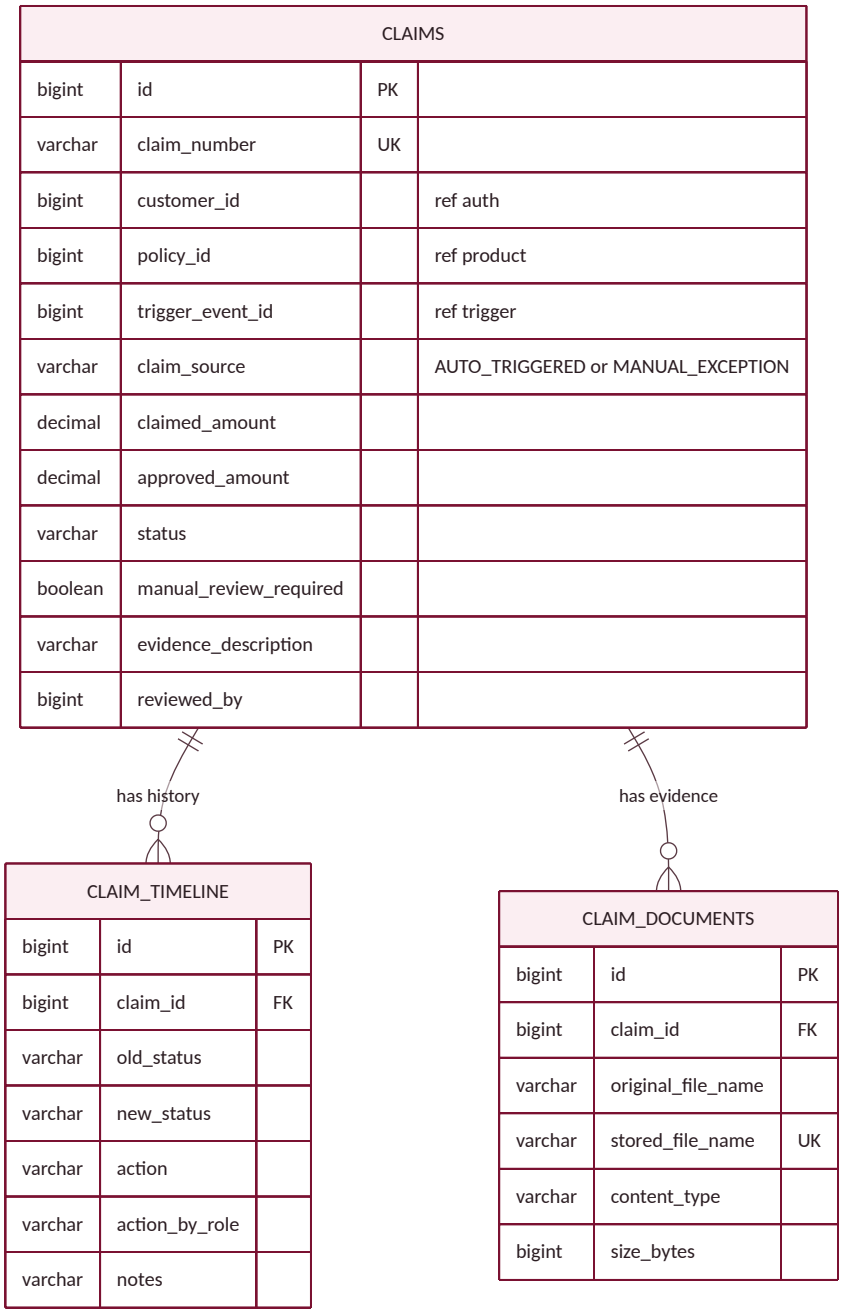


Figure 3.3: ER diagram — Claims Service (claimsdb)

3.5 Trigger Service Schema

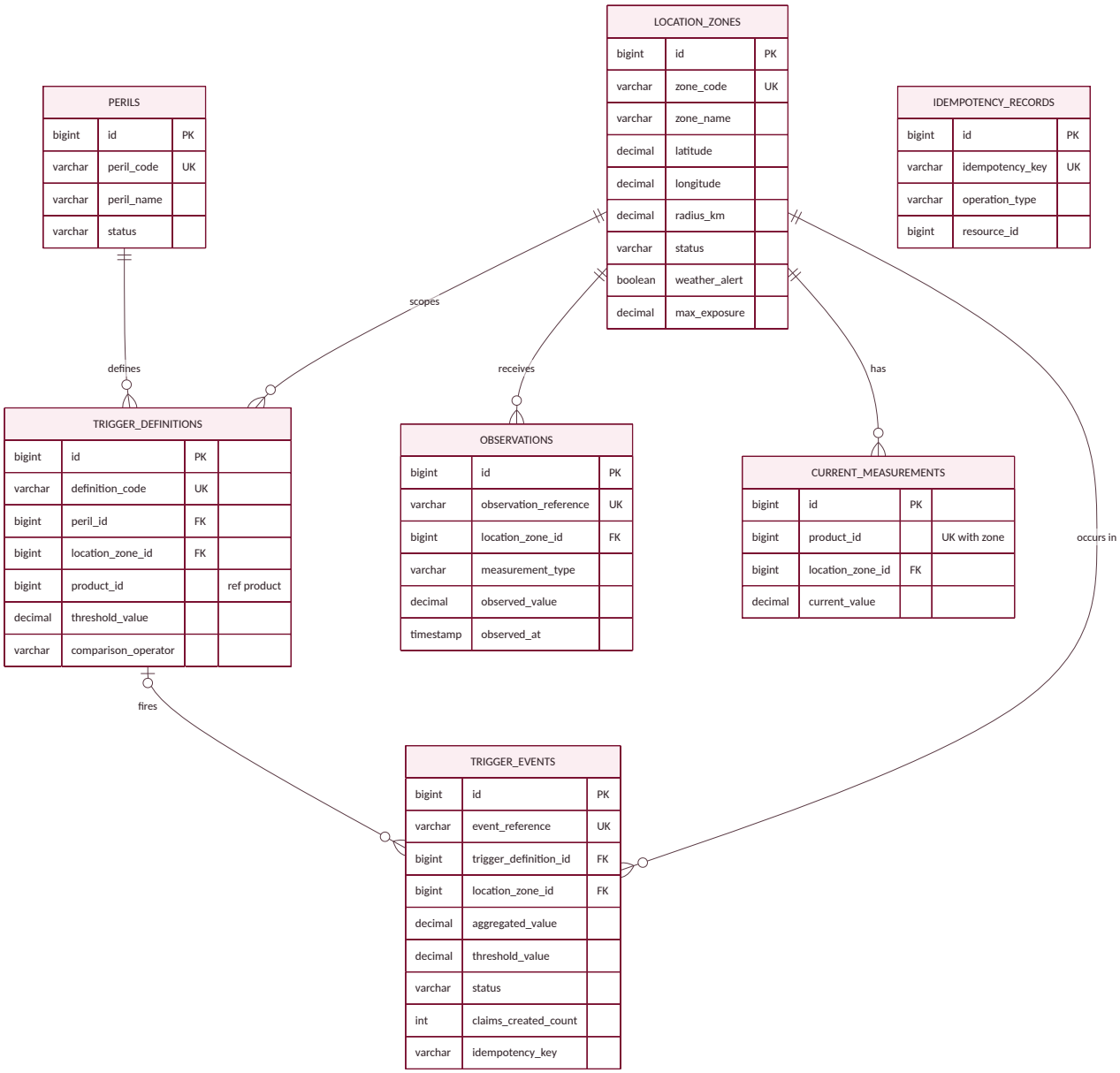


Figure 3.4: ER diagram — Trigger Service (triggerdb)

3.6 Financial and Notification Service Schemas

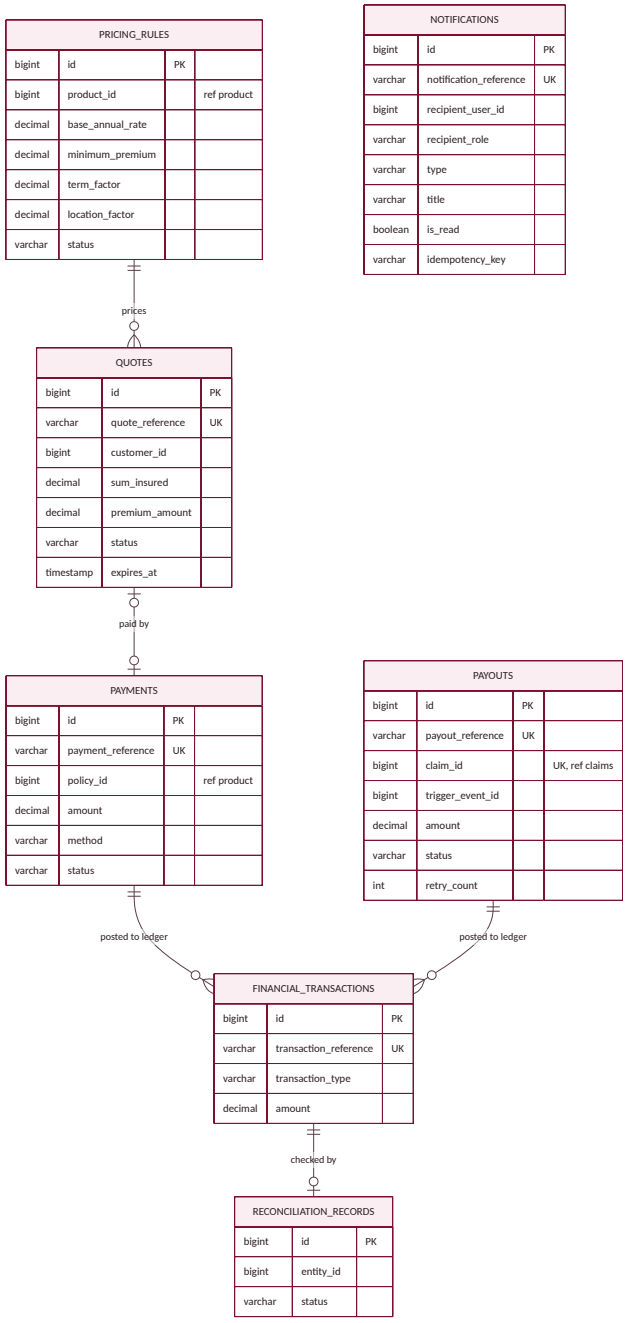


Figure 3.5: ER diagram — Financial Service (financialdb) and Notification Service (notificationdb)

3.7 Cross-Service References

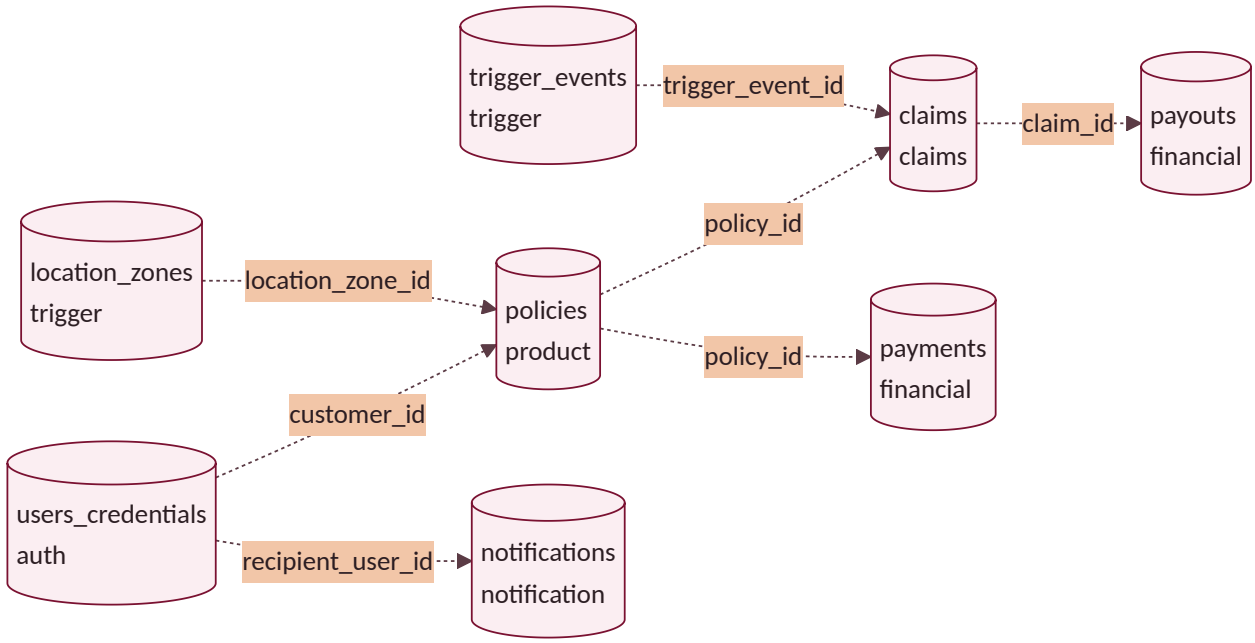


Figure 3.6: Logical references between service databases (by identifier, no physical foreign key)

3.8 Data Integrity Measures

- **Unique business keys** — policy, claim, payment, payout, quote, event and notification references are `UNIQUE`.
- **One payout per claim** — `payouts.claim_id` is unique, so a claim can never be paid twice.
- **One live value per product and zone** — `UNIQUE(product_id, location_zone_id)` and `CHECK(current_value ≥ 0)` on `current_measurements`.
- **Idempotency** — `idempotency_records.idempotency_key` and `notifications.idempotency_key` make repeated requests harmless.
- **Audit trails** — `policy_status_history`, `product_threshold_history` and `claim_timeline` record who changed what, and when.
- **Window check** — `CHECK(window_end ≥ window_start)` on trigger events.

CHAPTER 4

4. UML Diagrams

4.1 Use Case Diagram

Figure 4.1 shows the four human actors and the *Trigger Engine*, a system actor. Customers manage their own cover and claims. Underwriters, claims officers and administrators each work in a separate workspace. Simulating a weather reading *includes* trigger detection, which creates the claims automatically.

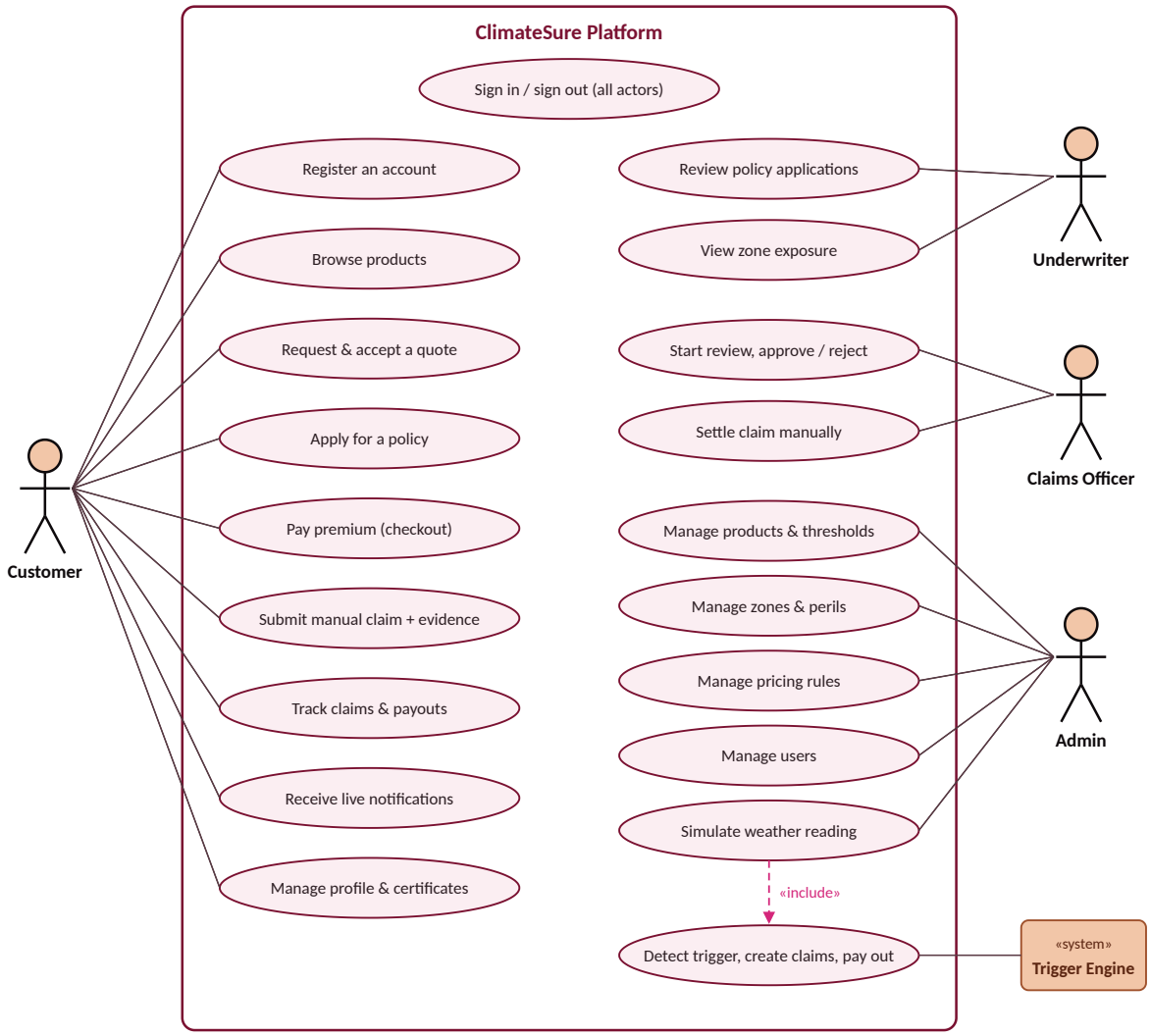


Figure 4.1: Use case diagram

4.2 Class Diagrams

Figure 4.2 shows the core domain model across all services. Solid associations exist inside one service (with a foreign key). Dashed dependencies cross service boundaries and are held only as identifiers.

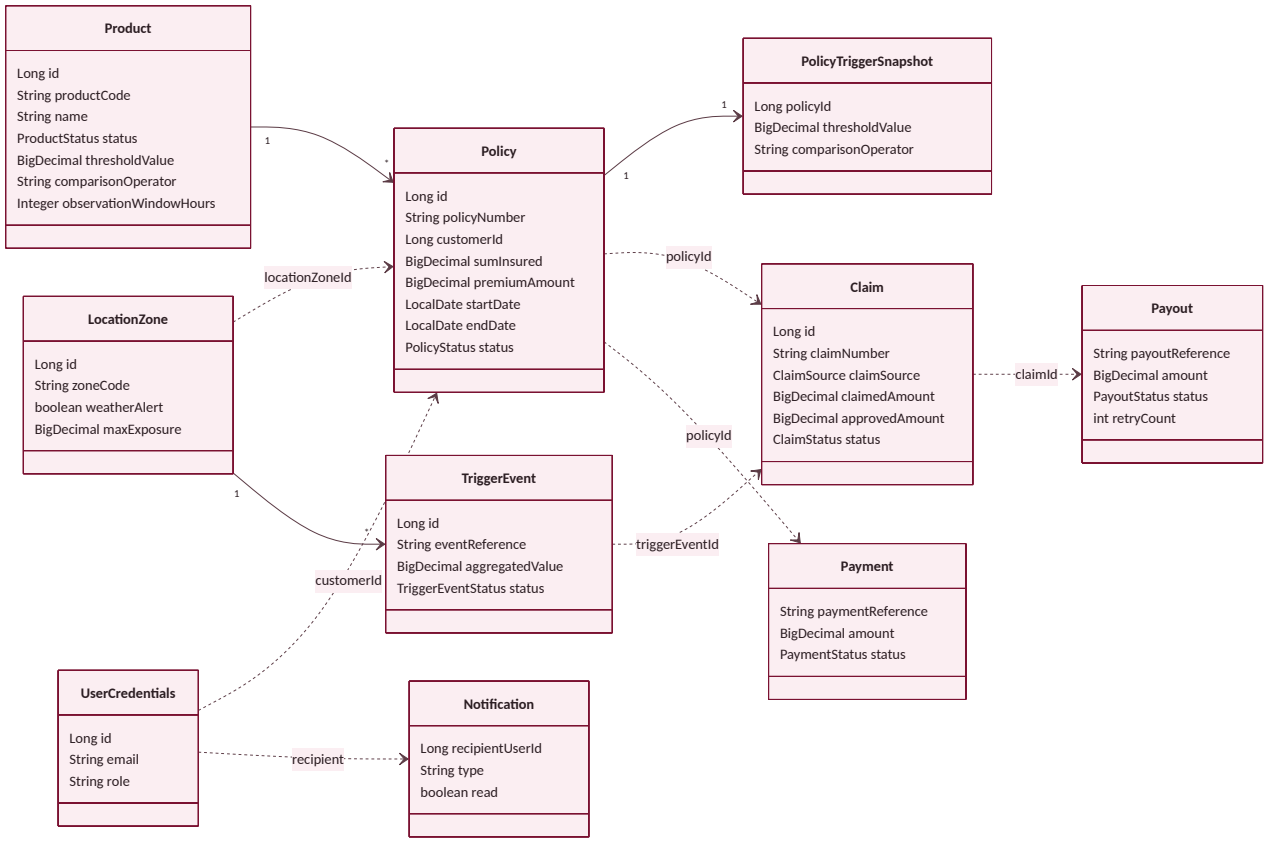


Figure 4.2: Domain class diagram

Each service uses the same layered structure: a functional **Router** maps paths to a **Handler**, the handler validates the DTO and calls a **Service**, and the service uses reactive **Repositories** and **Clients** for other services. Figure 4.3 shows this for the Claims Service.

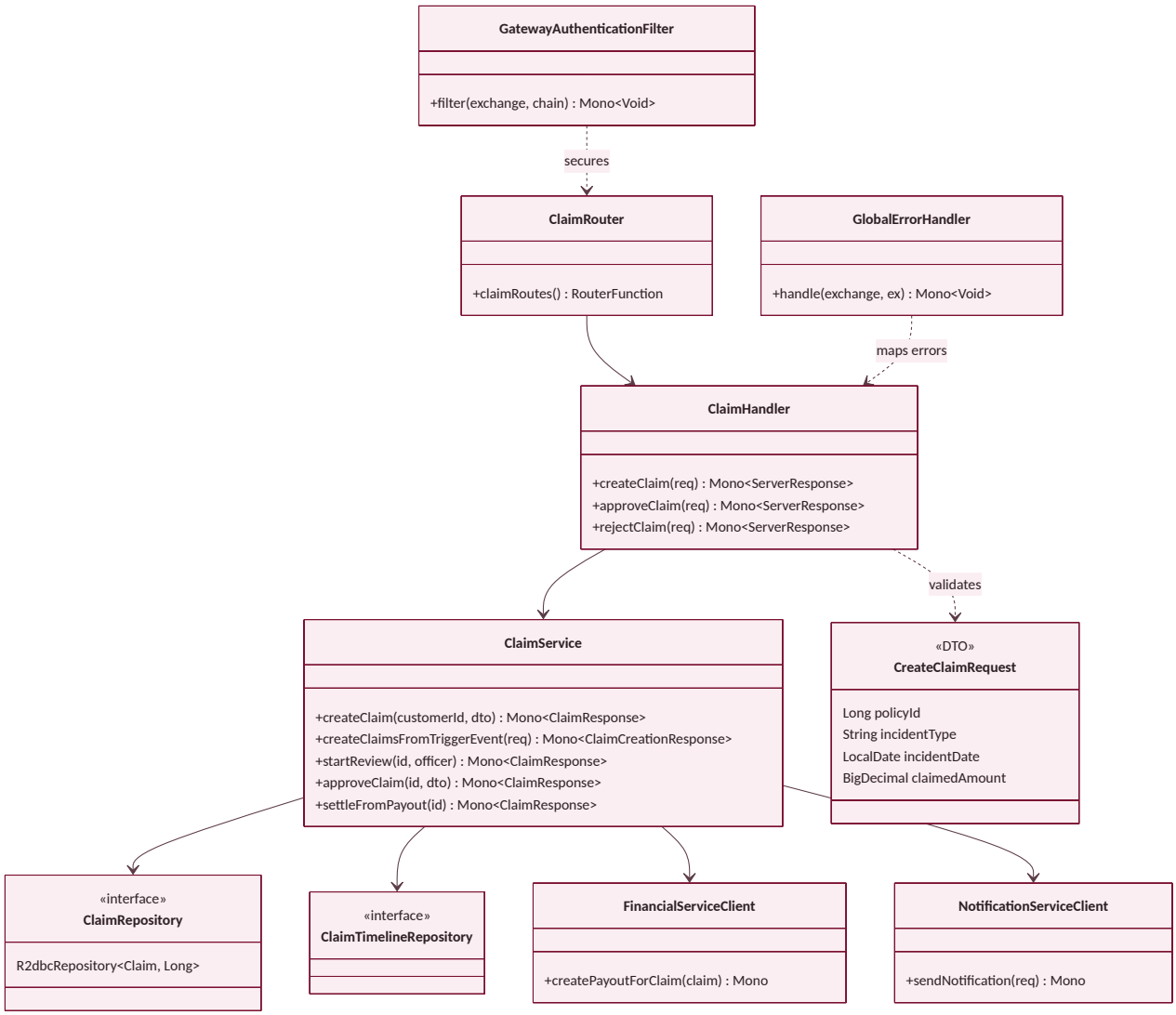


Figure 4.3: Layered class diagram — Claims Service

4.3 Sequence Diagrams

Login and authenticated request

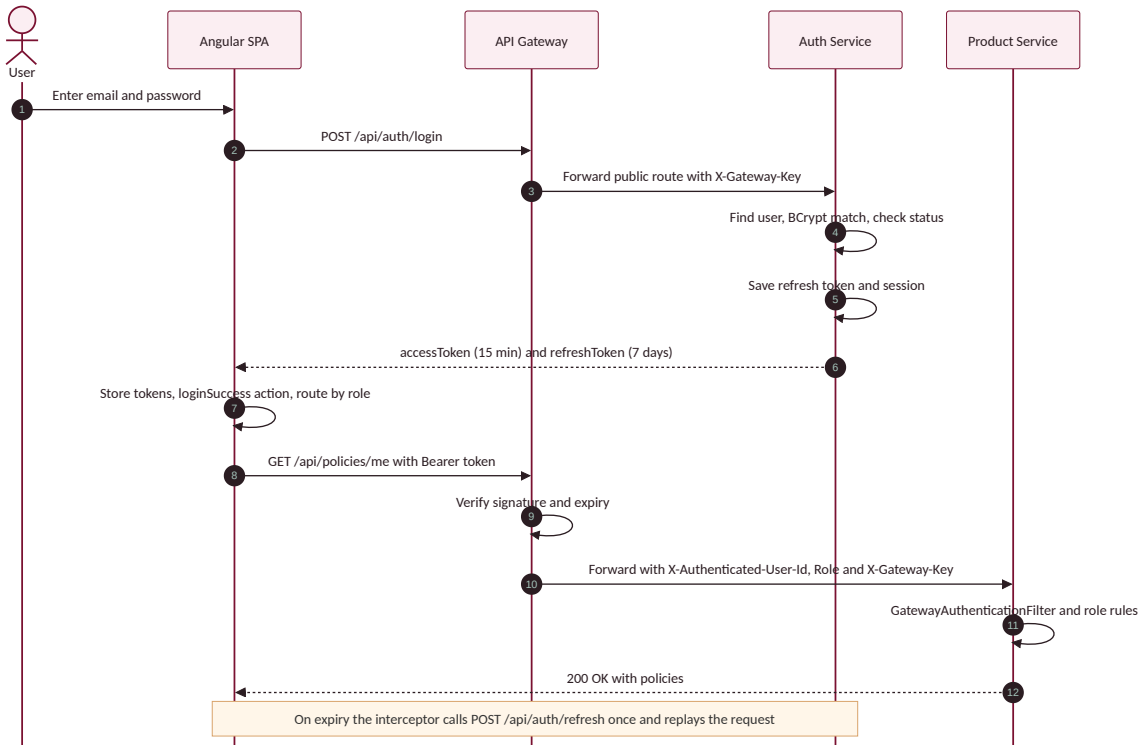


Figure 4.4: Sequence diagram — login, token use and refresh

Policy purchase

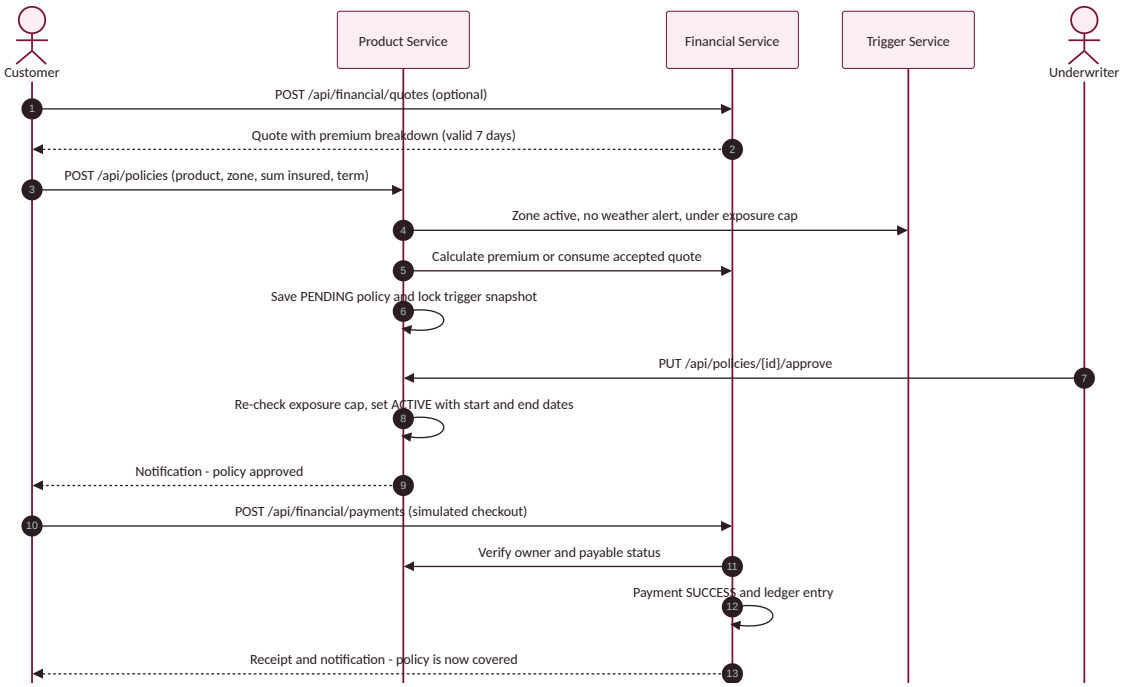


Figure 4.5: Sequence diagram — quote, application, underwriting and premium payment

Automatic parametric claim and payout (core flow)

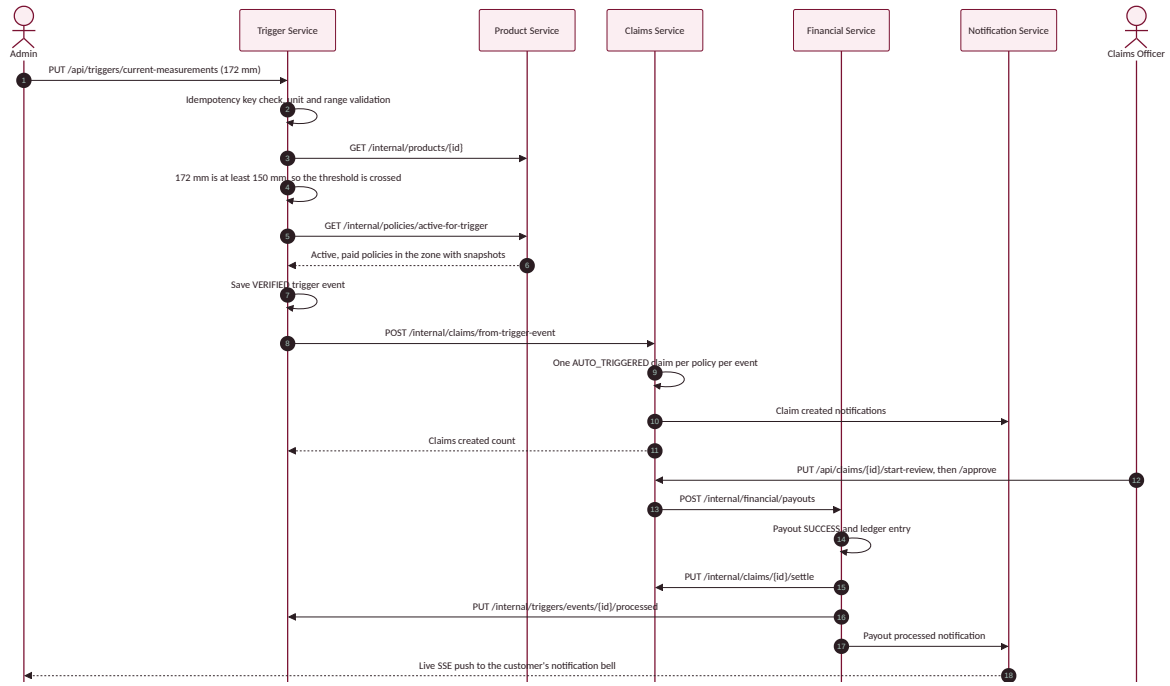


Figure 4.6: Sequence diagram — from weather reading to settled claim

4.4 State Diagrams

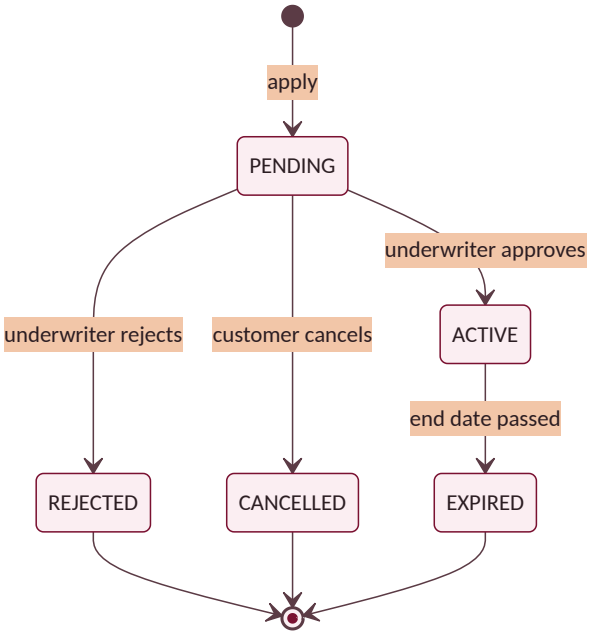


Figure 4.7: Policy life-cycle

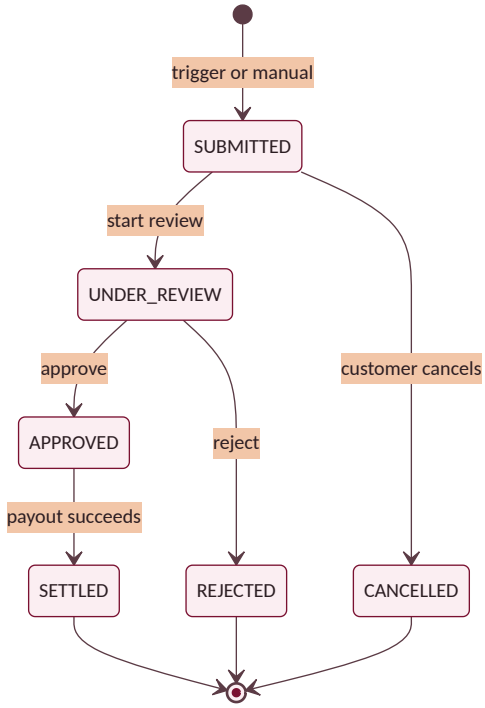


Figure 4.8: Claim life-cycle

Table 4.1: Life-cycles of other entities

Entity	Life-cycle
Quote	CREATED → ACCEPTED → CONSUMED · CREATED/ACCEPTED → CANCELLED · CREATED → EXPIRED (after 7 days)
Payment	SUCCESS → REFUNDED (admin)
Payout	CREATED → SUCCESS · FAILED → retried (max 3) → SUCCESS
Trigger event	VERIFIED on creation → PROCESSED when the payout succeeds
Zone / Peril	ACTIVE ↔ INACTIVE · ARCHIVED → restore → INACTIVE · weather alert on/off
Pricing rule	DRAFT → ACTIVE ↔ INACTIVE · INACTIVE → ARCHIVED → restore
User profile	ACTIVE ↔ INACTIVE ↔ SUSPENDED (inactive and suspended users cannot log in)

CHAPTER 5

5. User Journey & Flow

5.1 User Roles

Table 5.1: User roles and landing pages

Role	Goal	Home page
Customer	Buy cover, pay premiums, follow claims and receive payouts	/customer/dashboard
Underwriter	Approve or reject policy applications within the zone risk limits	/underwriter/dashboard
Claims Officer	Review claims, approve or reject them, and settle	/claims-officer/dashboard
Admin	Configure products, thresholds, zones, perils and prices, simulate weather, manage users	/admin/dashboard

5.2 Customer Journey

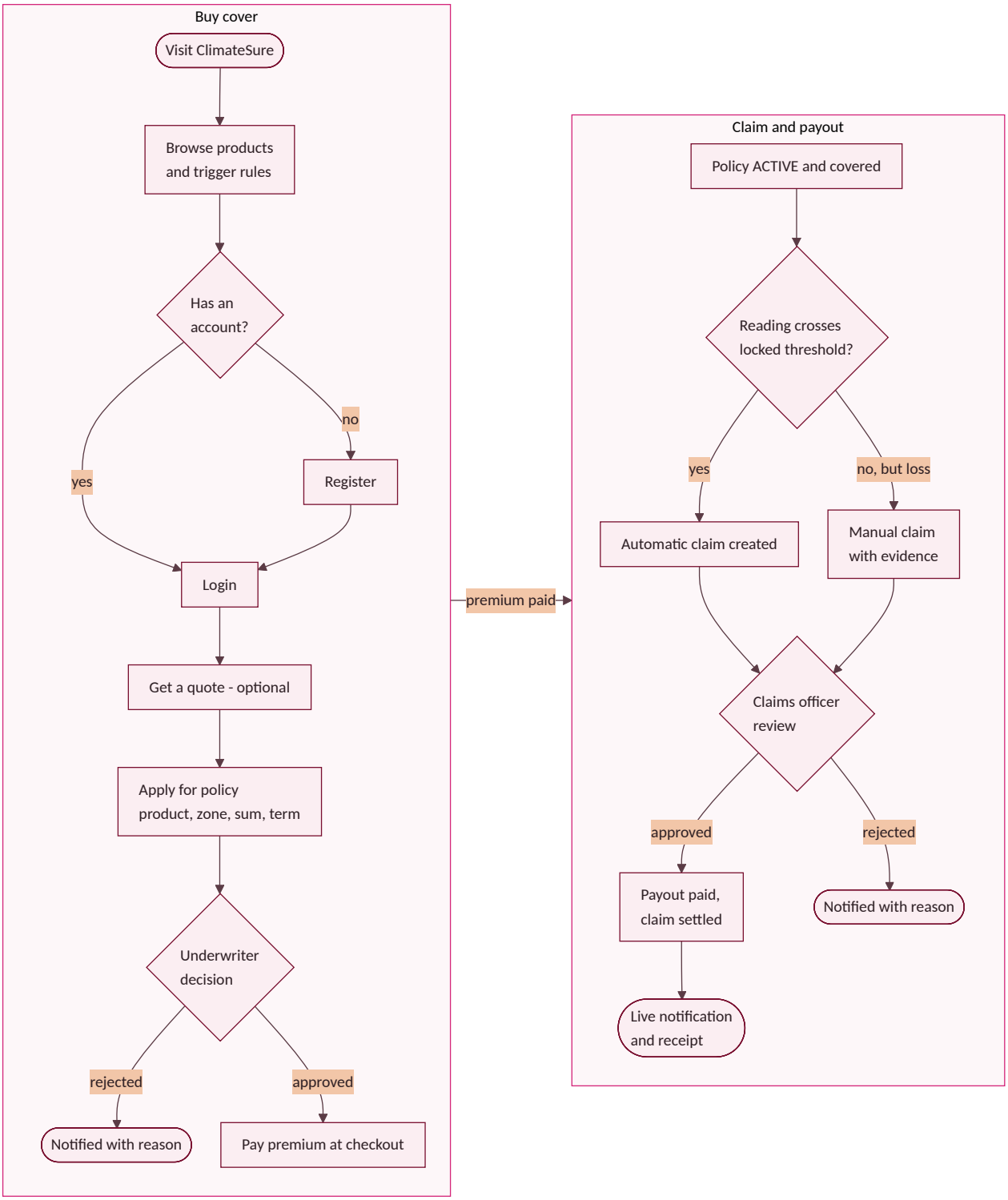


Figure 5.1: Customer journey flow

5.3 Staff Journeys

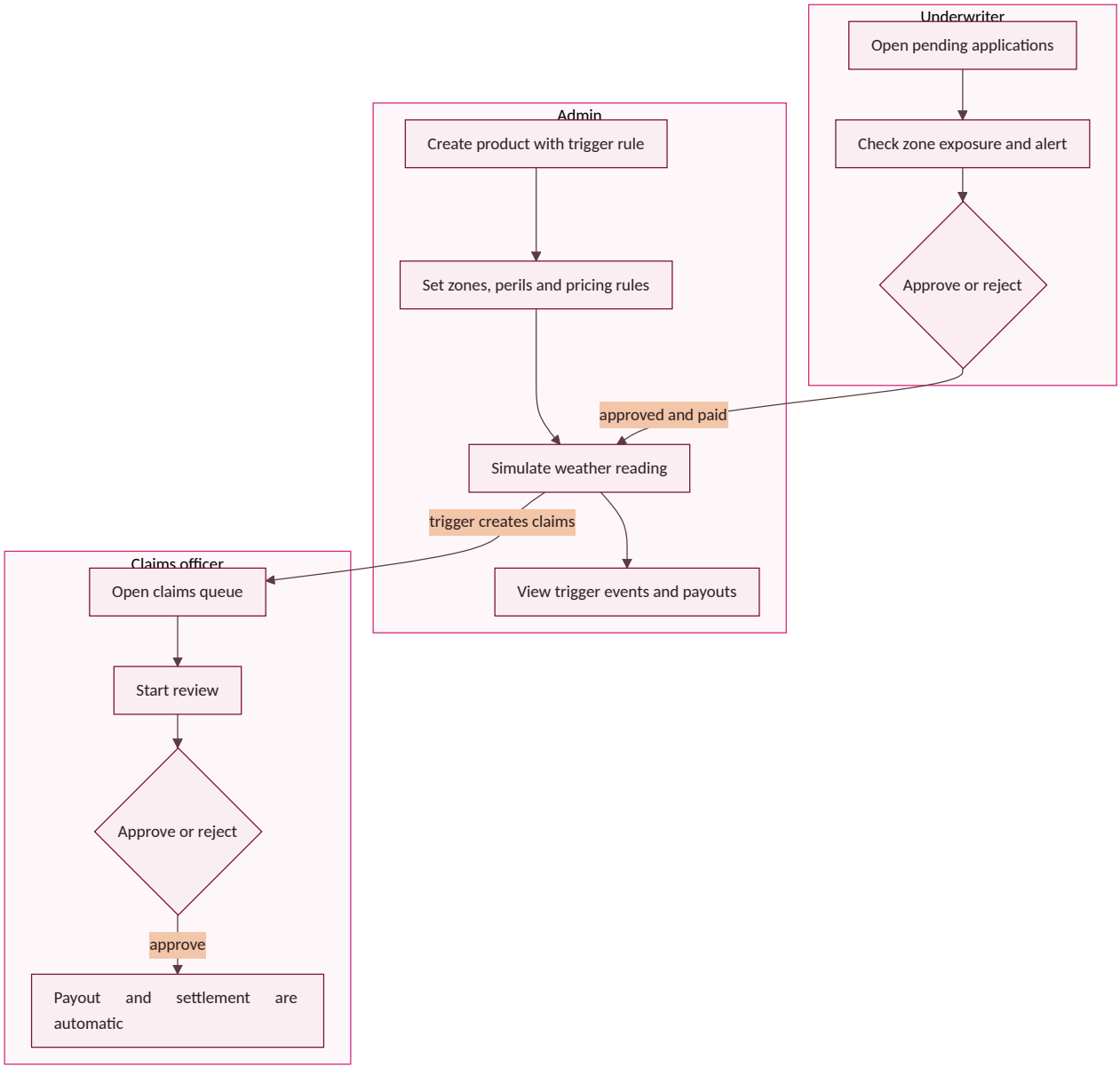


Figure 5.2: Staff journeys and hand-offs

5.4 Journey Catalogue

The full user-journey mapping (kept in `docs/ClimateSure_User_Journey_Mapping.xlsx` of the backend repository) documents 18 journeys step by step, with the pages, API calls and database tables used at each step.

Table 5.2: User journey catalogue

ID	Journey	Actor(s)
UJ-01	Customer registration, login and logout	Customer (guest)
UJ-02	Browse product catalogue (public)	Guest / Customer

ID	Journey	Actor(s)
UJ-03	Customer profile management	Customer
UJ-04	Get a quote and accept it	Customer
UJ-05	Apply for a policy (standard product)	Customer
UJ-06	Custom (customer-defined) parametric policy	Customer
UJ-07	Underwriting review (approve / reject policy)	Underwriter (or Admin)
UJ-08	Premium payment (simulated checkout)	Customer, Admin
UJ-09	Policy servicing (view, certificate, renew, auto-renew, cancel)	Customer
UJ-10	Admin product set-up and trigger threshold management	Admin
UJ-11	Admin reference data: zones, perils, pricing rules	Admin
UJ-12	Automatic parametric claim via measurement console (primary flow)	Admin, System, Customer
UJ-13	Customer manual claim (exception) with evidence	Customer
UJ-14	Claims officer review (start, approve → payout, reject, settle)	Claims Officer (or Admin)
UJ-15	Payout disbursement and receipt	Admin, Customer
UJ-16	In-app notifications and real-time push	All roles
UJ-17	Admin user management and dashboards	Admin
UJ-18	Background system jobs	System schedulers

5.5 Frontend Page Map

Table 5.3: Frontend page map by role

Area	Pages (routes)
Public	/welcome, /products, /products/:id, /login, /register
Customer	dashboard, products, quotes, policies, policies/new, policies/:id, payments, claims, claims/new, claims/:id, notifications, zone-map, profile
Underwriter	dashboard, policies, policies/:id, history, zone-exposure, notifications, profile
Claims officer	dashboard, claims, claims/:id, history, notifications, profile
Admin	dashboard, products (new, edit, threshold), product-thresholds, users, users/:id, policies, claims, payments, payouts, pricing-rules, zones, zone-exposure, perils, current-measurements (trigger simulator), trigger-events, notifications

CHAPTER 6

6. Functional Requirements & Business Rules

6.1 Functional Requirements

Table 6.1: Functional requirements

ID	Module	Requirement
FR-01	Accounts	Users can register as customers and log in with email and password.
FR-02	Accounts	The system issues a short-lived access token and a long-lived refresh token, and refreshes the session automatically.
FR-03	Accounts	Customers can maintain a profile, segment and address. Admins can activate, deactivate or suspend any user.
FR-04	Products	Admins can create, edit, activate, deactivate and archive products with sum-insured and term limits.
FR-05	Products	Every product has a machine-checkable trigger rule. Every change to it is recorded in threshold history.
FR-06	Quotes & pricing	Customers can request a quote that shows the full premium calculation. A quote can be accepted, cancelled or expire after 7 days.
FR-07	Quotes & pricing	Admins can manage pricing rules (base annual rate, term factor, zone factor, minimum premium).
FR-08	Policies	Customers can apply for a policy for an active product in an active zone, or define a custom parametric policy.
FR-09	Policies	Underwriters can approve or reject applications with a reason. Approval re-checks the zone exposure cap and weather alert.
FR-10	Policies	Customers can cancel pending applications, download a PDF certificate, renew and enable auto-renewal.
FR-11	Payments	Customers pay the premium through a simulated checkout (UPI, card, net banking). Each policy can be paid only once.
FR-12	Payments	Admins can view all payments and refund successful ones.
FR-13	Triggers	Admins can manage zones (with weather alerts and exposure caps) and perils, and enter the current value per product and zone.
FR-14	Triggers	When a value crosses the threshold, the system records a VERIFIED trigger event and creates claims for every active, paid policy in that zone.
FR-15	Claims	Customers can file a manual exception claim with an evidence description and up to three files, and cancel it while it is submitted.
FR-16	Claims	Claims officers can start a review and approve (with an amount) or reject (with a reason). The claim timeline records every step.
FR-17	Payouts	An approved claim is paid immediately, the claim is settled and the trigger event is marked processed.

ID	Module	Requirement
FR-18	Payouts	Failed payouts are retried automatically up to three times. Customers can download a PDF payout receipt.
FR-19	Notifications	Every important event creates an in-app notification for the right user or role.
FR-20	Notifications	New notifications are pushed live to the browser using Server-Sent Events. Users can mark notifications as read or delete them.
FR-21	Dashboards	Each role has a dashboard with key figures. Admins see financial metrics, trigger activity and zone exposure.

6.2 Non-Functional Requirements

Table 6.2: Non-functional requirements

ID	Category	Requirement
NFR-01	Security	All non-public endpoints require a valid JWT. Roles are checked at the gateway and again in each service. Passwords are stored as BCrypt hashes.
NFR-02	Scalability	Services are stateless and reactive, can be scaled independently and are discovered through Eureka.
NFR-03	Reliability	Trigger processing and notifications are idempotent. Failed payouts are retried. Reconciliation checks the ledger daily.
NFR-04	Performance	Non-blocking I/O end to end (WebFlux, R2DBC, WebClient). Lists are paginated and filtered in the frontend.
NFR-05	Usability	Responsive layout for desktop and phone. Indian number and date formats. Clear empty, loading and error states.
NFR-06	Maintainability	Layered code (router, handler, service, repository), DTO validation and a uniform error format.
NFR-07	Observability	Rolling log files per service, gateway access log, actuator health, info and metrics endpoints.
NFR-08	Accuracy	Money is stored as exact decimals (BigDecimal, 2 decimal places).

6.3 Business Rules

Table 6.3: Business rules enforced in the services

Area	Rule
Accounts	Email must be unique. Inactive or suspended accounts cannot log in or refresh their tokens.
Products	Only ACTIVE products can be sold. Threshold changes never affect existing policies (snapshot protection).
Policies	Sum insured and term must be within the product limits. Only one PENDING application per product per customer.
Policies	The zone must be ACTIVE, not under a weather alert, and within its exposure cap, both when applying and when approving.

Area	Rule
Policies	A rejection needs a reason (max 500 characters). Only PENDING applications can be cancelled.
Quotes	An accepted quote must belong to the customer, match product, zone, sum insured and term exactly, and not be expired.
Payments	The customer must own the policy. A policy can be paid only once. Only successful payments can be refunded.
Claims	The policy must be ACTIVE, owned by the customer and the premium paid. The incident date must fall inside the policy period.
Claims	The claimed amount must not exceed the sum insured. The approved amount must not exceed the claimed amount.
Claims	Status order: SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED. Only one claim per policy per trigger event.
Payouts	The claim must be APPROVED with an amount above zero. One payout per claim. Only FAILED payouts can be retried.
Triggers	The unit must suit the measurement. Percentages are 0–100. Temperature is −100 to 100 °C. Other values cannot be negative.
Triggers	Only paid, active policies in the affected zone receive claims. The same idempotency key is never processed twice.

6.4 Premium Calculation

The Financial Service calculates the premium from the active pricing rule of the product:

```
premium = sumInsured × baseAnnualRate × (termMonths ÷ 12) × termFactor × zoneFactor
premium = max(premium, minimumPremium)            // rounded to 2 decimals

Example: 1,50,000 × 3.5% × 12/12 × 1.0 × 1.0 = ₹ 5,250
```

CHAPTER 7

7. API Documentation

7.1 Conventions

- Base URL for all clients: `http://localhost:8080` (API Gateway). Request and response bodies are JSON.
- Authentication: `Authorization: Bearer <accessToken>` on every non-public endpoint.
- Path identifiers must be positive whole numbers. Lists are returned as JSON arrays.
- Status codes: `200` OK, `201` Created, `400` validation error, `401` missing or invalid token, `403` wrong role or not the owner, `404` not found, `409` conflict (duplicate or wrong state), `503` dependent service unavailable.
- Internal endpoints (`/internal/**`) are only callable between services with the gateway key and are not routed by the gateway.

7.2 Endpoint Summary

Table 7.1: Number of endpoints per service

Service	Public	Secured	Internal	Total
Auth Service	3	7	0	10
Product & Policy Service	4	28	5	37
Claims Service	0	22	3	25
Trigger Service	0	26	5	31
Financial Service	0	24	5	29
Notification Service	0	10	1	11
Total	**7**	**117**	**19**	**143**

7.3 Endpoint Reference

Tables 7.2 to 7.7 list every endpoint exposed through the gateway with the roles allowed to call it.

Table 7.2: Auth Service endpoints

Method	Path	Roles	Description
POST	`/api/auth/register`	PUBLIC	Register a new customer account
POST	`/api/auth/login`	PUBLIC	Authenticate with email + password (BCrypt)
POST	`/api/auth/refresh`	PUBLIC	Exchange a valid refresh token for a new access/refresh token pair (token rotation)
POST	`/api/auth/logout`	AUTHENTICATED	Revoke the supplied refresh token and end the current session

Method	Path	Roles	Description
GET	/api/auth/me	AUTHENTICATED	Return the signed-in user (id, name, email, role, customerId)
GET	/api/users/me	AUTHENTICATED	Get my profile (personal details, segment, addresses)
PUT	/api/users/me	CUSTOMER	Create/update my profile (name, phone, DOB, segment, address)
GET	/api/users	ADMIN	List all users, filterable by status and customer segment
GET	/api/users/{userId}	ADMIN, UNDERWRITER, CLAIMS_OFFICER	Get a single user's profile (staff lookup of a customer)
PUT	/api/users/{userId}/status	ADMIN	Change a user's status (ACTIVE / SUSPENDED / INACTIVE)

Table 7.3: Product & Policy Service endpoints

Method	Path	Roles	Description
POST	/api/policies/{policyId}/renew	CUSTOMER	Renew an ACTIVE/EXPIRED policy: creates a new PENDING application linked via renewedFromPolicyId
PUT	/api/policies/{policyId}/auto-renew	CUSTOMER	Turn auto-renew on/off
GET	/api/policies/{policyId}/certificate	CUSTOMER	Download my policy certificate (PDF)
GET	/api/policies/admin/{policyId}/certificate	ADMIN, UNDERWRITER	Download any policy certificate (PDF)
GET	/api/policies/zones/active	AUTHENTICATED	Active location zones (proxied from Trigger-Svc) for the policy application form
GET	/api/policies/admin/all	ADMIN, UNDERWRITER	All policies, filter by status / customerId
GET	/api/policies/admin/{policyId}	ADMIN, UNDERWRITER	Any policy by id (staff view)
GET	/api/policies/review/pending	UNDERWRITER, ADMIN	Underwriting queue: all PENDING policy applications
GET	/api/policies/review/history	UNDERWRITER, ADMIN	Approve/reject decisions made by the calling underwriter
GET	/api/policies/me	CUSTOMER	My policies (optional status filter)
GET	/api/policies/{policyId}	CUSTOMER	One of my policies
POST	/api/policies	CUSTOMER	Apply for a policy on an ACTIVE product
POST	/api/policies/custom	CUSTOMER	Customer-defined parametric cover: creates a customer-defined product (code CUSTOM-<customerId>-XXXX) with the...
PUT	/api/policies/{policyId}/cancel	CUSTOMER	Cancel my PENDING application (PENDING -> CANCELLED)

Method	Path	Roles	Description
PUT	/api/policies/{policyId}/approve	UNDERWRITER, ADMIN	Approve a PENDING application: status ACTIVE, start date (today or day after renewed policy ends), end date = start...
PUT	/api/policies/{policyId}/reject	UNDERWRITER, ADMIN	Reject a PENDING application with a mandatory reason (<=500 chars)
GET	/api/policies/{policyId}/history	CUSTOMER, UNDERWRITER, ADMIN	Status history of a policy
GET	/api/policies/{policyId}/snapshot	CUSTOMER, UNDERWRITER, ADMIN	Trigger snapshot captured when the policy was issued (threshold, operator, window...)
GET	/api/products	PUBLIC	List ACTIVE insurance products (public catalogue)
GET	/api/products/category/{category}	PUBLIC	List ACTIVE products in a category (WEATHER, FLOOD, AGRICULTURE, ...)
GET	/api/products/measurement-status	AUTHENTICATED	Public view of current measurement vs threshold per product/zone
GET	/api/products/{productId}	PUBLIC	Get one ACTIVE product with coverages and parametric trigger description
POST	/api/products	ADMIN	Create a product in DRAFT with limits (sum insured, term), coverages and parametric trigger config (peril,...
PUT	/api/products/{productId}	ADMIN	Update product details and trigger configuration (trigger must stay complete)
PUT	/api/products/{productId}/status	ADMIN	Change product status (DRAFT/ACTIVE/INACTIVE/ARCHIVED)
DELETE	/api/products/{productId}	ADMIN	Archive (soft-delete) a product when it has no open policies
GET	/api/products/admin/all	ADMIN	List all products in every status (admin catalogue)
GET	/api/products/admin/measurement-status	ADMIN	Measurement console: every product x zone with current value, threshold, operator and whether it is breached
GET	/api/products/admin/{productId}	ADMIN	Get any product (any status) with trigger sync status
GET	/api/products/admin/{productId}/threshold-history	ADMIN	History of trigger threshold changes for a product (old/new values, reason, who, when)
PUT	/api/products/admin/{productId}/threshold	ADMIN	Change the product's trigger threshold / operator / window
GET	/api/policies/admin/zone-exposure	ADMIN	Total sum insured of active policies per zone against the zone exposure cap

Table 7.4: Claims Service endpoints

Method	Path	Roles	Description
POST	/api/claims/{claimId}/documents	CUSTOMER	Upload evidence files (photos/PDF, multipart) to my claim
GET	/api/claims/{claimId}/documents	CUSTOMER	List documents on my claim
GET	/api/claims/{claimId}/documents/{documentId}	CUSTOMER	Download one of my claim documents
GET	/api/claims/review/{claimId}/documents	CLAIMS_OFFICER, ADMIN	List documents on any claim under review
GET	/api/claims/review/{claimId}/documents/{documentId}	CLAIMS_OFFICER, ADMIN	Download a claim document for review
GET	/api/claims/admin/all	ADMIN	All claims, filter by status / customerId
GET	/api/claims/admin/{claimId}	ADMIN	Any claim (admin)
GET	/api/claims/review/manual	CLAIMS_OFFICER, ADMIN	Queue of MANUAL_EXCEPTION claims needing investigation
GET	/api/claims/review/manual/{claimId}	CLAIMS_OFFICER, ADMIN	One manual exception claim
GET	/api/claims/review/submitted	CLAIMS_OFFICER, ADMIN	Claims queue: SUBMITTED + UNDER_REVIEW claims
GET	/api/claims/review/history	CLAIMS_OFFICER, ADMIN	Decisions made by the calling officer
GET	/api/claims/review/{claimId}	CLAIMS_OFFICER, ADMIN	Any claim for review
GET	/api/claims/me	CUSTOMER	My claims (optional status filter)
GET	/api/claims/{claimId}	CUSTOMER	One of my claims
POST	/api/claims	CUSTOMER	Customer files a manual claim (MANUAL_EXCEPTION): policy must be ACTIVE and owned, premium paid, customer confirms...
PUT	/api/claims/{claimId}/cancel	CUSTOMER	Cancel my SUBMITTED claim (SUBMITTED -> CANCELLED)
PUT	/api/claims/{claimId}/start-review	CLAIMS_OFFICER, ADMIN	Pick up a claim (SUBMITTED -> UNDER_REVIEW)
PUT	/api/claims/{claimId}/approve	CLAIMS_OFFICER, ADMIN	Approve (UNDER_REVIEW -> APPROVED) with approvedAmount (>0 and <= claimedAmount)
PUT	/api/claims/{claimId}/reject	CLAIMS_OFFICER, ADMIN	Reject (UNDER_REVIEW -> REJECTED) with reason
PUT	/api/claims/{claimId}/settle	CLAIMS_OFFICER, ADMIN	Settle (APPROVED -> SETTLED) with optional notes
GET	/api/claims/{claimId}/timeline	CUSTOMER	Timeline of status changes on my claim
GET	/api/claims/admin/{claimId}/timeline	ADMIN	Timeline of any claim

Table 7.5: Trigger Service endpoints

Method	Path	Roles	Description
GET	/api/triggers/zones	ADMIN, UNDERWRITER, CLAIMS_OFFICER	All location zones (filter by state)
GET	/api/triggers/zones/active	AUTHENTICATED	Active zones (used by customer forms and maps)
GET	/api/triggers/zones/{zoneId}	ADMIN, UNDERWRITER, CLAIMS_OFFICER, CUSTOMER	One zone (customers see the zone name on their policy page)
POST	/api/triggers/zones	ADMIN	Create a location zone (code, name, state, district, lat/long, radius)
PUT	/api/triggers/zones/{zoneId}	ADMIN	Update a zone
PUT	/api/triggers/zones/{zoneId}/status	ADMIN	Change zone status (ACTIVE / INACTIVE / ARCHIVED)
DELETE	/api/triggers/zones/{zoneId}	ADMIN	Archive a zone (blocked while ACTIVE policies exist in it)
PUT	/api/triggers/current-measurements	ADMIN	Measurement console: record the current value for a product in a zone (idempotency key)
GET	/api/triggers/current-measurements	ADMIN, UNDERWRITER, CLAIMS_OFFICER, CUSTOMER	Current measurement snapshots per product/zone
GET	/api/triggers/perils	ADMIN, UNDERWRITER, CLAIMS_OFFICER	All perils
GET	/api/triggers/perils/{perilId}	ADMIN, UNDERWRITER, CLAIMS_OFFICER	One peril
POST	/api/triggers/perils	ADMIN	Create a peril (code, name, type, measurement type/unit)
PUT	/api/triggers/perils/{perilId}	ADMIN	Update a peril
PUT	/api/triggers/perils/{perilId}/status	ADMIN	Change peril status
DELETE	/api/triggers/perils/{perilId}	ADMIN	Archive a peril
GET	/api/triggers/definitions/{definitionId}	ADMIN, UNDERWRITER, CLAIMS_OFFICER	One trigger definition (shown on a trigger event)
GET	/api/triggers/events	ADMIN, UNDERWRITER, CLAIMS_OFFICER	All trigger events (created VERIFIED by the measurement console; PROCESSED after the payout)
GET	/api/triggers/events/{eventId}	ADMIN, UNDERWRITER, CLAIMS_OFFICER	One trigger event with its definition, peril and zone

Method	Path	Roles	Description
GET	/api/triggers/measurements/zones	AUTHENTICATED	Latest weather readings per zone for the zone map
GET	/api/triggers/perils/archived	ADMIN	Archived perils (for the "Show archived" panel)
PUT	/api/triggers/perils/{perilId}/restore	ADMIN	Restore an archived peril; it comes back INACTIVE (409 if not archived)
GET	/api/triggers/zones/archived	ADMIN	Archived zones (for the "Show archived" panel)
PUT	/api/triggers/zones/{zoneId}/restore	ADMIN	Restore an archived zone; it comes back INACTIVE (409 if not archived)
PUT	/api/triggers/zones/{zoneId}/alert	ADMIN	Turn a weather alert for a zone on (with message, max 500) or off
PUT	/api/triggers/zones/{zoneId}/exposure-cap	ADMIN	Set the zone exposure cap (empty = no cap, otherwise at least 1)
PUT	/api/triggers/measurements/current	ADMIN	Header measurement panel: set a live value for a measurement in every active zone (held 60 min before the demo feed...

Table 7.6: Financial Service endpoints

Method	Path	Roles	Description
POST	/api/financial/pricing-rules	ADMIN	Create a pricing rule for product (+ optional zone): base annual rate, term factor, location factor, minimum...
GET	/api/financial/pricing-rules	ADMIN	List all pricing rules
GET	/api/financial/pricing-rules/{ruleId}	ADMIN	Get a pricing rule
PUT	/api/financial/pricing-rules/{ruleId}	ADMIN	Update a pricing rule (re-validates product and zone)
PUT	/api/financial/pricing-rules/{ruleId}/status	ADMIN	Change rule status (DRAFT / ACTIVE / INACTIVE / ARCHIVED)
DELETE	/api/financial/pricing-rules/{ruleId}	ADMIN	Archive a pricing rule
POST	/api/financial/quotes	CUSTOMER	Get a quote: validates product limits and zone, finds applicable ACTIVE pricing rule
GET	/api/financial/quotes/me	CUSTOMER	My quotes (auto-expires stale ones)
GET	/api/financial/quotes/{quoteId}	CUSTOMER	One of my quotes
PUT	/api/financial/quotes/{quoteId}/accept	CUSTOMER	Accept a CREATED quote (-> ACCEPTED)
PUT	/api/financial/quotes/{quoteId}/cancel	CUSTOMER	Cancel my quote (-> CANCELLED)
POST	/api/financial/payments	CUSTOMER	Pay the premium for my policy (mock card / bank / UPI)
GET	/api/financial/payments/me	CUSTOMER	My payments
GET	/api/financial/payments/{paymentId}	CUSTOMER, ADMIN	A payment (customer: own only)

Method	Path	Roles	Description
GET	/api/financial/payments/admin/all	ADMIN	All payments
PUT	/api/financial/payments/{paymentId}/refund	ADMIN	Refund a SUCCESS payment (-> REFUNDED)
POST	/api/financial/payouts	CLAIMS_OFFICER, ADMIN	Create a payout for an APPROVED claim (one payout per claim)
GET	/api/financial/payouts/me	CUSTOMER	My payouts
GET	/api/financial/payouts/{payoutId}	CUSTOMER, ADMIN	A payout (customer: own only)
GET	/api/financial/payouts/admin/all	ADMIN	All payouts
PUT	/api/financial/payouts/{payoutId}/retry	ADMIN	Retry an old FAILED payout: it is paid straight away
GET	/api/financial/admin/metrics	ADMIN	Financial KPIs (premium collected, payouts, counts by status) for the admin dashboard
GET	/api/financial/payouts/{payoutId}/receipt	CUSTOMER	Download my payout receipt (PDF)
GET	/api/financial/payouts/admin/{payoutId}/receipt	ADMIN	Download any payout receipt (PDF)

Table 7.7: Notification Service endpoints

Method	Path	Roles	Description
GET	/api/notifications/me	AUTHENTICATED	My in-app notifications, newest first (own, customer-id and role broadcasts)
GET	/api/notifications/me/unread	AUTHENTICATED	My unread notifications
GET	/api/notifications/me/unread-count	AUTHENTICATED	Unread badge count
GET	/api/notifications/me/stream	AUTHENTICATED (token in query)	Server-Sent Events stream of new notifications for the caller
PUT	/api/notifications/{notificationId}/read	AUTHENTICATED	Mark one of my notifications read
PUT	/api/notifications/me/read-all	AUTHENTICATED	Mark all my notifications read
DELETE	/api/notifications/{notificationId}	AUTHENTICATED	Delete one of my notifications
GET	/api/notifications/admin/all	ADMIN	All notifications (paged, filter by type)
PUT	/api/notifications/admin/{notificationId}/read	ADMIN	Mark any notification read
DELETE	/api/notifications/admin/{notificationId}	ADMIN	Delete any notification

7.4 Internal Endpoints

Table 7.8: Internal service-to-service endpoints

Method	Path	Owning service
GET	/internal/claims/{claimId}	Claims Service
POST	/internal/claims/from-trigger-event	Claims Service
PUT	/internal/claims/{claimId}/settle	Claims Service
POST	/internal/financial/pricing/calculate	Financial Service
POST	/internal/financial/quotes/{quoteId}/validate	Financial Service
PUT	/internal/financial/quotes/{quoteId}/consume	Financial Service
GET	/internal/financial/payments/policy/{policyId}/status	Financial Service
POST	/internal/financial/payouts	Financial Service
POST	/internal/notifications	Notification Service
GET	/internal/policies/active-for-trigger	Product & Policy Service
GET	/internal/policies/zone/{zoneId}/has-active	Product & Policy Service
GET	/internal/policies/{policyId}	Product & Policy Service
GET	/internal/products	Product & Policy Service
GET	/internal/products/{productId}	Product & Policy Service
GET	/internal/triggers/zones/{zoneId}	Trigger Service
GET	/internal/triggers/zones/active	Trigger Service
GET	/internal/triggers/measurements/current	Trigger Service
PUT	/internal/triggers/products/{productId}/definition	Trigger Service
PUT	/internal/triggers/events/{eventId}/processed	Trigger Service

7.5 Sample Requests and Responses

Login

```
POST /api/auth/login
Content-Type: application/json

{ "email": "akshaya@gmail.com", "password": "********" }

HTTP/1.1 200 OK
{
  "accessToken": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiIyNTciLCJlbWFpbCI6...",
  "refreshToken": "42254d22-c8a5-4228-...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": { "id": 257, "fullName": "Akshaya Chigullapally",
            "email": "akshaya@gmail.com", "role": "ROLE_CUSTOMER" }
}
```

Apply for a policy

```
POST /api/policies
Authorization: Bearer <customer token>

{ "productId": 1, "sumInsured": 150000, "termMonths": 12, "locationZoneId": 1 }

HTTP/1.1 201 Created
{ "id": 225, "policyNumber": "POL-2026-5044647B", "productCode": "WEATHER-001",
  "sumInsured": 150000.00, "premiumAmount": 5250.00, "status": "PENDING", ... }
```

Simulate a weather reading (admin)

```
PUT /api/triggers/current-measurements
Authorization: Bearer <admin token>

{ "productId": 1, "locationZoneId": 1, "currentValue": 172,
  "idempotencyKey": "3f6c1a1e-6f0b-4f9e-9a57-0d1f4c1b7a21" }

HTTP/1.1 200 OK
{ "matched": true, "productCode": "WEATHER-001", "measurementType": "RAINFALL",
  "measurementUnit": "mm", "currentValue": 172, "currentProductThresholdValue": 150.0,
  "comparisonOperator": "GREATER_THAN_OR_EQUAL", "triggerEventId": 129,
  "triggerEventStatus": "VERIFIED", "matchedPolicyCount": 6, "claimsCreatedCount": 6, ... }
```

Error response

```
HTTP/1.1 409 Conflict
{
  "timestamp": "2026-10-10T14:13:34.544",
  "status": 409,
  "error": "Conflict",
  "messages": ["A pending policy application already exists for this product"],
  "path": "/api/policies"
}
```

CHAPTER 8

8. Security & Role-Based Access Control

8.1 Security Overview

Security is applied in depth across three layers: the Angular client, the API Gateway and every microservice. The system is **stateless**. No server-side HTTP session exists. Every request carries a signed JSON Web Token, and every service decides independently whether the caller may perform the action.

Table 8.1: Security layers

Layer	Mechanism
Client (Angular)	authGuard, guestGuard and roleGuard on routes. authInterceptor attaches the token and refreshes it before expiry. errorInterceptor handles 401/403 and logs the user out when the refresh fails.
API Gateway	GatewayJwtFilter verifies the HMAC-SHA512 signature and expiry, rejects invalid tokens with 401, strips any X-Authenticated-* headers sent by the client and adds trusted user id, email and role headers plus the secret X-Gateway-Key. CORS allows only the SPA origin.
Microservices	Spring Security (WebFlux) with GatewayAuthenticationFilter: requests without the correct gateway key are refused. The role header becomes a Spring authority, and path rules in SecurityConfig (pathMatchers(…).hasAnyRole(…)) allow or deny each endpoint. Anything not listed is denied.
Service logic	Ownership checks (a customer can only see and change their own policies, claims, payments and documents), status checks and business rules.
Data	Passwords hashed with BCrypt. Refresh tokens stored, can be revoked on logout and expire after 7 days. Uploaded evidence is stored under random file names.

8.2 JSON Web Tokens

On login, the Auth Service issues an **access token** valid for 15 minutes and a **refresh token** valid for 7 days. The access token is signed with HMAC-SHA512 (JJWT library) using a secret shared by the Auth Service and the gateway. It contains:

```
Header : { "alg": "HS512" }
Payload: {
  "sub": "257",                       // user id
  "email": "akshaya@gmail.com",
  "role": "ROLE_CUSTOMER",
  "customerId": 257,
  "iat": 1791641601,                  // issued at
  "exp": 1791642501                   // expires 15 minutes later
}
```

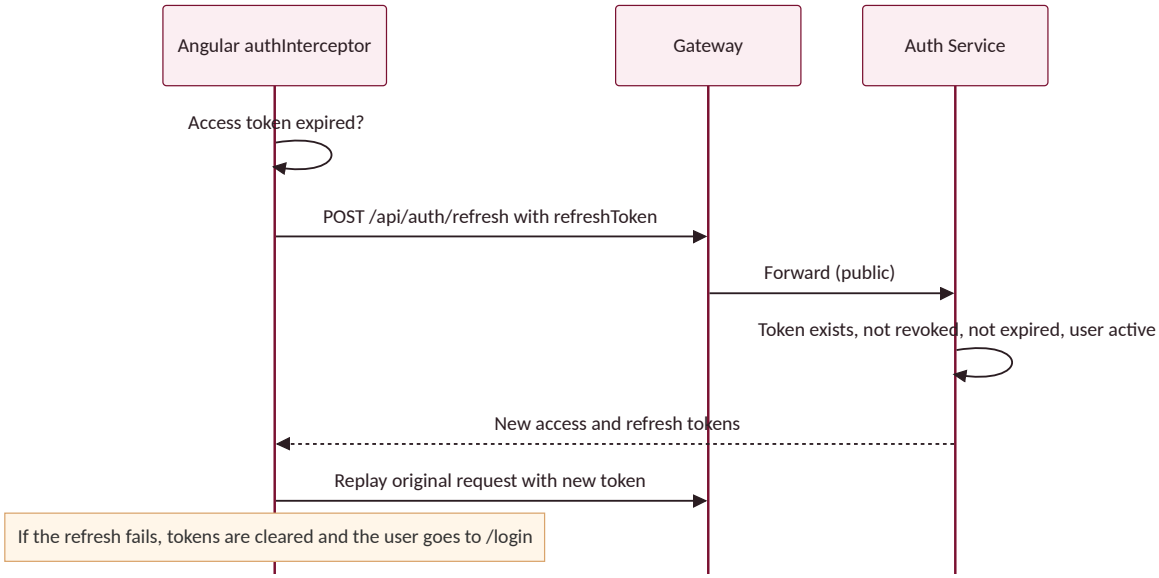


Figure 8.1: Token refresh flow

8.3 Spring Security Configuration

Each service declares its rules in a `SecurityWebFilterChain`. The `GatewayAuthenticationFilter` runs first and refuses (403) any request that does not carry the correct gateway key, so the `/internal/**` paths below are reachable only by other services. An extract from the Claims Service:

```
return http
        .csrf(ServerHttpSecurity.CsrfSpec::disable)
        .httpBasic(ServerHttpSecurity.HttpBasicSpec::disable)
        .formLogin(ServerHttpSecurity.FormLoginSpec::disable)
        .securityContextRepository(NoOpServerSecurityContextRepository.getInstance())
        .authorizeExchange(auth->auth
                .pathMatchers("/actuator","/actuator/**").permitAll()
                .pathMatchers(HttpMethod.POST,"/api/claims").hasRole("CUSTOMER")
                .pathMatchers(HttpMethod.GET,"/api/claims/me").hasRole("CUSTOMER")
                .pathMatchers(HttpMethod.PUT,
                        "/api/claims/{claimId:[0-9]+}/start-review",
                        "/api/claims/{claimId:[0-9]+}/approve",
                        "/api/claims/{claimId:[0-9]+}/reject",
                        "/api/claims/{claimId:[0-9]+}/settle"
                ).hasAnyRole("CLAIMS_OFFICER","ADMIN")
                .pathMatchers(HttpMethod.GET,"/api/claims/admin/all").hasRole("ADMIN")
                .pathMatchers(HttpMethod.POST,"/internal/claims/from-trigger-
event").permitAll()
                .anyExchange().denyAll())
        .addFilterAt(gatewayAuthenticationFilter,SecurityWebFiltersOrder.AUTHENTICATION)
        .build();
```

8.4 Role-Based Access Matrix

Table 8.2: Role-based access matrix

Capability	Customer	Underwriter	Claims Officer	Admin
Browse products, register, login	✔	✔	✔	✔
Request quotes, apply for and pay for policies	✔	—	—	—
View own policies, claims, payments, payouts	✔	—	—	—
Submit or cancel own claims, upload evidence	✔	—	—	—
Approve or reject policy applications	—	✔	—	✔
View all policies and zone exposure	—	✔	—	✔
Review, approve, reject and settle claims	—	—	✔	✔
View zones, perils and trigger events	—	✔	✔	✔
Manage products, thresholds, zones, perils, pricing	—	—	—	✔
Simulate weather readings	—	—	—	✔
Manage users, refunds, payout retries, metrics	—	—	—	✔
Own notifications and live stream	✔	✔	✔	✔

8.5 Additional Security Measures

- **Header spoofing protection**: a client sending `X-Authenticated-Role: ROLE_ADMIN` has it removed by the gateway.
- **No direct service access**: services reject any request without the gateway key, even on their own port.
- **Internal APIs hidden**: `/internal/**` paths are never routed by the gateway.
- **Account status**: suspended or inactive users cannot log in, and their refresh tokens are refused.
- **Logout**: the refresh token is revoked and the session marked inactive.
- **Upload safety**: evidence files are limited to images and PDFs of up to 5 MB, at most three per claim, and stored under random names.
- **SSE authentication**: the browser EventSource cannot set headers, so the stream accepts the token as an `access_token` query parameter, which the gateway validates in the same way.

CHAPTER 9

9. Validation, DTOs & Exception Handling

9.1 DTO Design

Every request and response body is a **Data Transfer Object** in the `dto` package of each service, separate from the persistence **models**. DTOs decouple the API contract from the database schema, hide internal fields and carry all input-validation annotations. Lombok (`@Data`, `@Builder`) keeps them concise.

```
@Data
@Builder
public class CreateClaimRequest{
    @NotNull(message="Policy id is required")
    @Positive(message="Policy id must be positive")
    private Long policyId;

    @NotBlank(message="Incident type is required")
    @Size(max=100,message="Incident type must not exceed 100 characters")
    private String incidentType;

    @NotNull(message="Incident date is required")
    @PastOrPresent(message="Incident date cannot be in the future")
    private LocalDate incidentDate;

    @NotNull(message="Claimed amount is required")
    @Positive(message="Claimed amount must be positive")
    @Digits(integer=13,fraction=2,message="Claimed amount can have at most 2 decimal places")
    private BigDecimal claimedAmount;

    @Pattern(regexp="(?i)^\s*(AUTO_TRIGGERED|MANUAL_EXCEPTION)\s*$",
             message="Claim source must be AUTO_TRIGGERED or MANUAL_EXCEPTION")
    private String claimSource;

    @Size(max=2000,message="Evidence description must not exceed 2000 characters")
    private String evidenceDescription;
    ...
}
```

9.2 Backend Validation

- **Bean Validation** (Jakarta Validation) annotations on DTOs: `@NotBlank`, `@NotNull`, `@Positive`, `@Size`, `@Pattern`, `@Email`, `@DecimalMin`, `@Digits`, `@Past`, `@PastOrPresent` and `@AssertTrue` for cross-field checks.
- Handlers run the validator before calling the service (`readAndValidate`). An empty body returns “Request body is required”.
- Path variables are parsed and must be positive numbers, otherwise an “Invalid … id” error is returned.
- Rules that depend on stored data (ownership, limits, duplicates, state transitions) are enforced in the service layer (see Table 6.3).

Table 9.1: Examples of DTO validation rules

Request	Field	Rule
Register	fullName / email / password	Required. 2–150 characters / valid email up to 150 / 8–100 characters
Update profile	phoneNumber	10 digits starting with 6, 7, 8 or 9
Update profile	postalCode / dateOfBirth	Exactly 6 digits / must be in the past
Create product	productCode	Required, up to 50 characters, letters, digits, _ and -
Apply for policy	sumInsured / termMonths	Positive, 2 decimals / 1–60 months
Reject policy or claim	reason	Required, up to 500 characters
Approve claim	approvedAmount	Required, greater than zero
Current measurement	currentValue	Not negative. Percentages ≤ 100
User status	status	ACTIVE, INACTIVE or SUSPENDED

9.3 Frontend Validation

Angular **reactive forms** mirror the backend rules with built-in and custom validators (for example phone-number, postal-code and past-date validators). Errors are shown next to the field while the user types, and the submit button stays disabled until the form is valid, so most mistakes never reach the server. The backend still re-validates every request because client checks can be bypassed.

9.4 Exception Handling

Each service has a custom exception hierarchy and a single `GlobalErrorHandler` that implements WebFlux's `ErrorWebExceptionHandler` (ordered before the default handler). It maps exceptions to HTTP status codes, logs them (warning for 4xx, error with stack trace for 5xx) and writes a uniform JSON body.

Table 9.2: Exception to HTTP status mapping

Exception	HTTP status	Typical cause
ValidationException, IllegalArgumentException	400 Bad Request	DTO validation failed, malformed id
UnauthorizedException / invalid JWT	401 Unauthorized	Missing, expired or tampered token
ForbiddenException	403 Forbidden	Wrong role or not the owner of the resource
NotFoundException	404 Not Found	Policy, claim or product does not exist
ConflictException	409 Conflict	Duplicate email, policy already paid, wrong status transition
DependentServiceException	503 Service Unavailable	A downstream service is down or timed out
Any other exception	500 Internal Server Error	Unexpected failure (logged with stack trace)

```
{
  "timestamp": "2026-10-10T14:13:54.111",
  "status": 400,
  "error": "Bad Request",
  "messages": ["Incident date cannot be before policy start date"],
  "path": "/api/claims"
}
```

The gateway adds a `ServiceUnavailableHandler` that returns a clean 503 when a service is not registered in Eureka. In the frontend, `errorInterceptor` and the NgRx effects turn these messages into toast notifications and inline form errors.

CHAPTER 10

10. Technology Stack & Project Structure

10.1 Technology Stack

Table 10.1: Technology stack

Layer	Technology	Use
Language	Java 17, TypeScript 5.9	Backend and frontend
Backend framework	Spring Boot 4.1, Spring WebFlux (Reactor, Netty)	Reactive REST services with functional routers
Persistence	Spring Data R2DBC, H2 (file mode)	Non-blocking database access, one database per service
Service discovery	Spring Cloud Netflix Eureka	Registry and client-side load balancing
Gateway	Spring Cloud Gateway (WebFlux)	Routing, JWT filter, CORS
Security	Spring Security, JJWT, BCrypt	Authentication, authorisation, password hashing
Validation	Jakarta Bean Validation (Hibernate Validator)	DTO validation
Utilities	Lombok, Jackson	Boilerplate reduction, JSON
Monitoring	Spring Boot Actuator, SLF4J / Logback	Health, info, metrics, rolling logs
Backend testing	JUnit 5, Mockito, Reactor StepVerifier, WebTestClient, JaCoCo	Unit and integration tests, coverage
Frontend framework	Angular 21.2 (standalone components, signals)	Single-page application
State management	NgRx 21 (store, effects, selectors)	Predictable shared state
Styling & maps	Tailwind CSS 4, Leaflet	Responsive UI, zone map
Frontend testing	Vitest 4 (with V8 coverage)	Component, service, store and guard tests
Build & tools	Maven, Angular CLI, npm, Git & GitHub	Build, dependency and version control

10.2 Backend Project Structure

```
ParametricInsurance/
├── EurekaServer/            service registry (:8761)
├── apigateway/              Spring Cloud Gateway (:8080)
│   └── filter/              GatewayJwtFilter, RequestLoggingFilter, ServiceUnavailableHandler
├── authservice/             (:8081)
├── productservice/          (:8082)
├── claimsservice/           (:8083)
├── notificationservice/     (:8085)
├── triggerservice/          (:8087)
├── financialservice/        (:8088)
└── docs/                    FEATURES.md, VALIDATIONS.md, user-journey workbook

<each service>/src/main/java/org/example/<service>/
├── config/       SecurityConfig, WebClientConfig
├── filter/       GatewayAuthenticationFilter
├── router/       functional RouterFunction beans (URL → handler)
├── handler/      request parsing, DTO validation, response building
├── service/      business logic, schedulers
├── client/       WebClient clients for other services
├── repository/   R2dbcRepository interfaces
├── model/        persistence entities and enums
├── dto/          request / response objects
└── exception/    custom exceptions, GlobalErrorHandler, ErrorResponse
src/main/resources/  application.yaml, schema.sql
```

10.3 Frontend Project Structure

```
InsuranceFrontend/src/app/
├── core/
│   ├── constants/      api-endpoints.ts, roles.ts, storage-keys.ts
│   ├── guards/         auth.guard.ts, guest.guard.ts, role.guard.ts
│   ├── interceptors/   auth.interceptor.ts, error.interceptor.ts
│   ├── models/         auth-user, jwt-claims
│   └── services/       token-storage, page-title strategy
├── features/
│   ├── components/     52 page components (customer-*, underwriter-*, claims-officer-*, admin-
*, public)
│   ├── services/       15 HTTP services (auth, policy, claim, payment, payout, notification-
push, ...)
│   ├── state/          13 NgRx feature slices (actions, reducer, effects, selectors)
│   ├── dtos/           TypeScript interfaces matching the backend DTOs
│   └── utils/          formatters and helpers
├── layouts/            public, auth and app layouts, header with notification bell
├── shared/             reusable components (dialogs, toasts, status badges, tables)
├── app.routes.ts       lazy, guarded routes per role
└── app.config.ts       providers: router, HttpClient + interceptors, NgRx store and effects
```

10.4 Deployment View

Table 10.2: Deployment of the components

Component	How it runs	Address
Eureka Server	java -jar eureka-server.jar	http://localhost:8761
Six microservices	java -jar <service>.jar (each writes ./data and ./logs)	ports 8081–8088
API Gateway	java -jar apigateway.jar	http://localhost:8080
Angular SPA	npm install, then ng serve (development) or ng build (production)	http://localhost:4200

CHAPTER 11

11. Testing & Code Coverage

11.1 Testing Strategy

Table 11.1: Test levels and tools

Level	Backend	Frontend
Unit	Services and handlers with Mockito mocks. Reactive results verified with Reactor StepVerifier.	Services with HttpTestingController, reducers, selectors, effects, guards, interceptors, pipes and utilities.
Component / slice	@WebFluxTest and WebTestClient for routers and security rules. @DataR2dbcTest for repositories.	Component tests with TestBed and provideMockStore.
Integration	@SpringBootTest with an in-memory H2 database for complete request flows.	—
End-to-end (manual)	The full stack was started and the trigger-to-payout flow was run through the gateway (Chapter 13).	Every role's pages exercised in Chromium.

11.2 Backend Results

Table 11.2: Backend test results and JaCoCo coverage

Service	Tests	Failures	Line	Instruction	Branch
API Gateway	5	0	36.8%	34.8%	9.4%
Auth Service	105	0	91.9%	91.9%	74.1%
Product & Policy	107	0	79.2%	82.2%	50.7%
Claims Service	94	0	79.2%	77.6%	62.6%
Trigger Service	54	0	87.2%	87.9%	53.2%
Financial Service	110	0	82.3%	84.0%	65.8%
Notification Service	102	0	97.7%	96.5%	93.4%
Total	**577**	**0**			

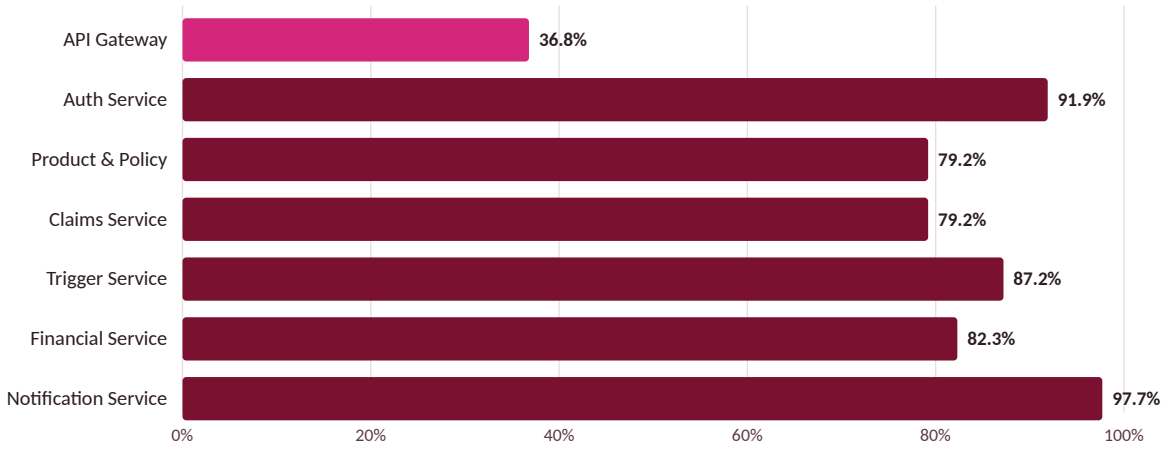


Figure 11.1: Backend line coverage by service (JaCoCo)

All backend tests pass. The business services (Auth, Product, Claims, Trigger, Financial and Notification) have between 79% and 98% line coverage. The API Gateway is mostly configuration and filters that are exercised by the end-to-end runs rather than unit tests, so its unit coverage is lower.

11.3 Frontend Results

Table 11.3: Frontend test results (Vitest, V8 coverage)

Metric	Value
Test files	124
Tests	293 passed, 0 failed
Statements	65.74% (9,558 / 14,537)
Branches	60.07% (2,788 / 4,641)
Functions	51.50% (1,218 / 2,365)
Lines	67.12% (8,273 / 12,324)

Frontend tests cover the logic that would break silently: authentication and token refresh, guards, every NgRx reducer, selector and effect, the HTTP services and the main page components. Pure layout templates are checked visually instead (Chapter 13).

11.4 Sample Test

A unit test from `ClaimServiceTest` that checks a business rule with Mockito and StepVerifier:

```
@Test
void approveClaim_whenAmountExceedsClaimedAmount_shouldReturnValidationException() {
    Claim claim=claim(1L,ClaimStatus.UNDER_REVIEW);
    ApproveClaimRequest request=mock(ApproveClaimRequest.class);

    when(request.getApprovedAmount())
            .thenReturn(new BigDecimal("60000"));
    when(claimRepository.findById(1L))
            .thenReturn(Mono.just(claim));

    StepVerifier.create(claimService.approveClaim(50L,1L,request))
            .expectErrorSatisfies(error->{
                assertThat(error).isInstanceOf(ValidationException.class);
                assertThat(error.getMessage()).isEqualTo(
                        "Approved amount cannot exceed claimed amount"
                );
            })
            .verify();

    verify(claimRepository,never()).save(any());
}
```

CHAPTER 12

12. Logging & Monitoring

12.1 Logging

- **SLF4J with Logback** (via Lombok `@Slf4j`) in every service: about 310 log statements at DEBUG, INFO, WARN and ERROR level.
- **Rolling log files** per service, for example `./logs/claims-service.log`, with a maximum file size of 10 MB and 7 days of history, alongside console output.
- **Configurable levels**: the root logger is INFO and the application packages are DEBUG by default, overridable with the `APP_LOG_LEVEL` environment variable.
- **Gateway access log**: `RequestLoggingFilter` logs method, path, final status and duration for every request: INFO for 2xx/3xx, WARN for 4xx, ERROR for 5xx.
- **Error logging**: `GlobalErrorHandler` logs every rejected request with method, path and status, and adds the stack trace for server errors.
- **Business events**: trigger detection, claim creation, payouts, retries and reconciliation results are logged with their references (event, claim and payout numbers), which makes a flow traceable across services.

Log lines recorded during the result run in Chapter 13 (timestamps and thread names shortened):

```
[API-GATEWAY]   INFO RequestLoggingFilter : POST /api/policies -> 201 (164 ms)
[API-GATEWAY]   WARN RequestLoggingFilter : POST /api/policies -> 409 (56 ms)
[PRODUCT]       INFO PolicyService        : Policy 225 status PENDING -> ACTIVE by 2
[FINANCIAL]     INFO PaymentService       : Payment PAY-F6CE3A4FFA0F479C paid for policyId=225
amount=5250.00
[API-GATEWAY]   INFO RequestLoggingFilter : PUT /api/triggers/current-measurements -> 200 (2873
ms)
[CLAIMS]        INFO ClaimService         : Creating automatic claims for triggerEventId=129
across 6 policies
[CLAIMS]        INFO ClaimService         : Claim CLM-2026-8A534BE2 CLAIM_SUBMITTED: null ->
SUBMITTED
[FINANCIAL]     INFO PayoutService        : Creating payout for claim CLM-2026-8A534BE2
amount=150000.00
[FINANCIAL]     INFO PayoutService        : Payout PO-F400A0DF7959405A paid automatically
[FINANCIAL]     INFO ClaimsServiceClient  : Claim 132 settled after payout PO-F400A0DF7959405A
[TRIGGER]       INFO TriggerEventService  : Mark event processed [trigger event]: eventId=129
[PRODUCT]       WARN GlobalErrorHandler   : POST /api/policies rejected with 409: A pending
policy
                                            application already exists for this product
```

12.2 Monitoring

Table 12.1: Monitoring facilities

Tool	What it provides
Spring Boot Actuator	/actuator/health, /actuator/info and /actuator/metrics on every service (the gateway also exposes /actuator/mappings). Health is reachable through the gateway, e.g. /trigger/actuator/health.
Eureka dashboard	http://localhost:8761 lists every registered instance and its status (UP / DOWN).
Admin dashboard	Business monitoring in the UI: policies, claims, payouts, premium income, trigger events and zone exposure.
Financial metrics API	GET /api/financial/admin/metrics: total premium collected, total payout amount, and counts of quotes, payments and payouts by status.
Reconciliation job	Daily check that every payment and payout has a ledger entry, recorded in reconciliation_records.
Trigger event log	Every verified event stores matched policies, claims created and notifications sent.

CHAPTER 13

13. Results

The complete system was started locally: the Eureka Server, the API Gateway, the six microservices and the Angular development server. All seven applications registered with Eureka. The customer account **akshaya@gmail.com** (Akshaya Chigullapally) was then taken through the full parametric flow:

1. applied for a *Weather Protection Plan* (₹1,50,000, Hyderabad Central) and a *Flood Protection Plan* (₹2,50,000, Mumbai Central);
2. the underwriter approved both applications and the customer paid the premiums (₹5,250 and ₹11,250);
3. the administrator entered a rainfall reading of 172 mm for Hyderabad Central, above the 150 mm threshold;
4. the trigger engine created claim **CLM-2026-8A534BE2** automatically; the claims officer approved it; the payout of ₹1,50,000 was paid at once and the claim was settled;
5. the customer also filed a manual exception claim for local waterlogging (₹40,000), which is waiting for review, and has one more application pending.

The following screenshots were taken from the running application.

13.1 Public Pages

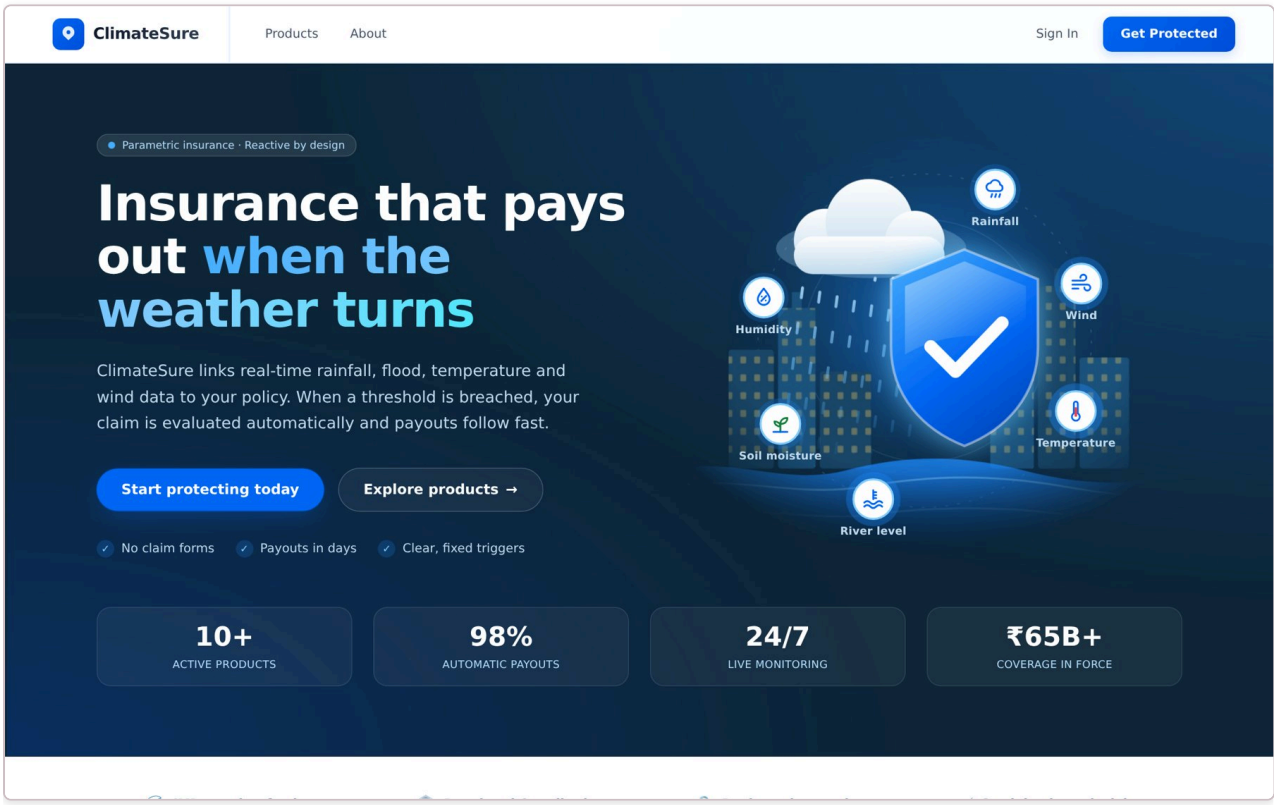


Figure 13.1: Landing page

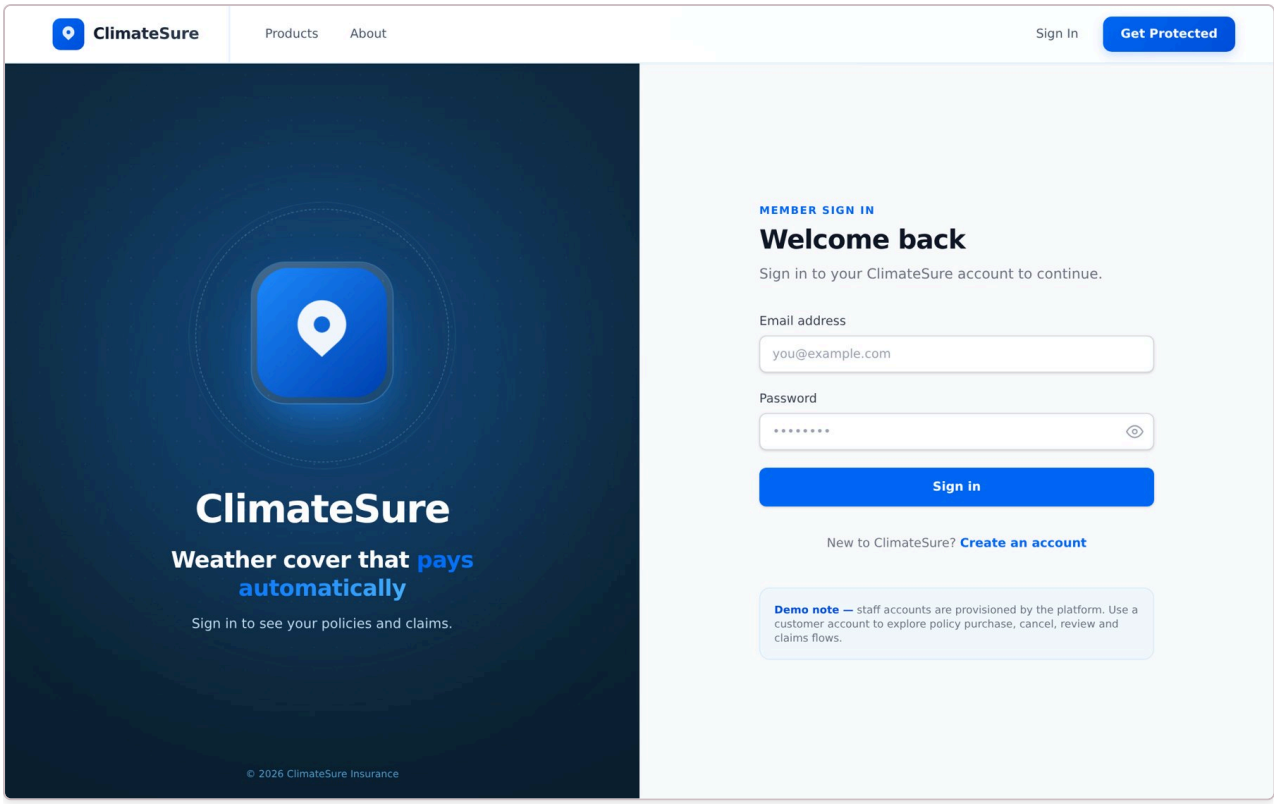


Figure 13.2: Sign-in page

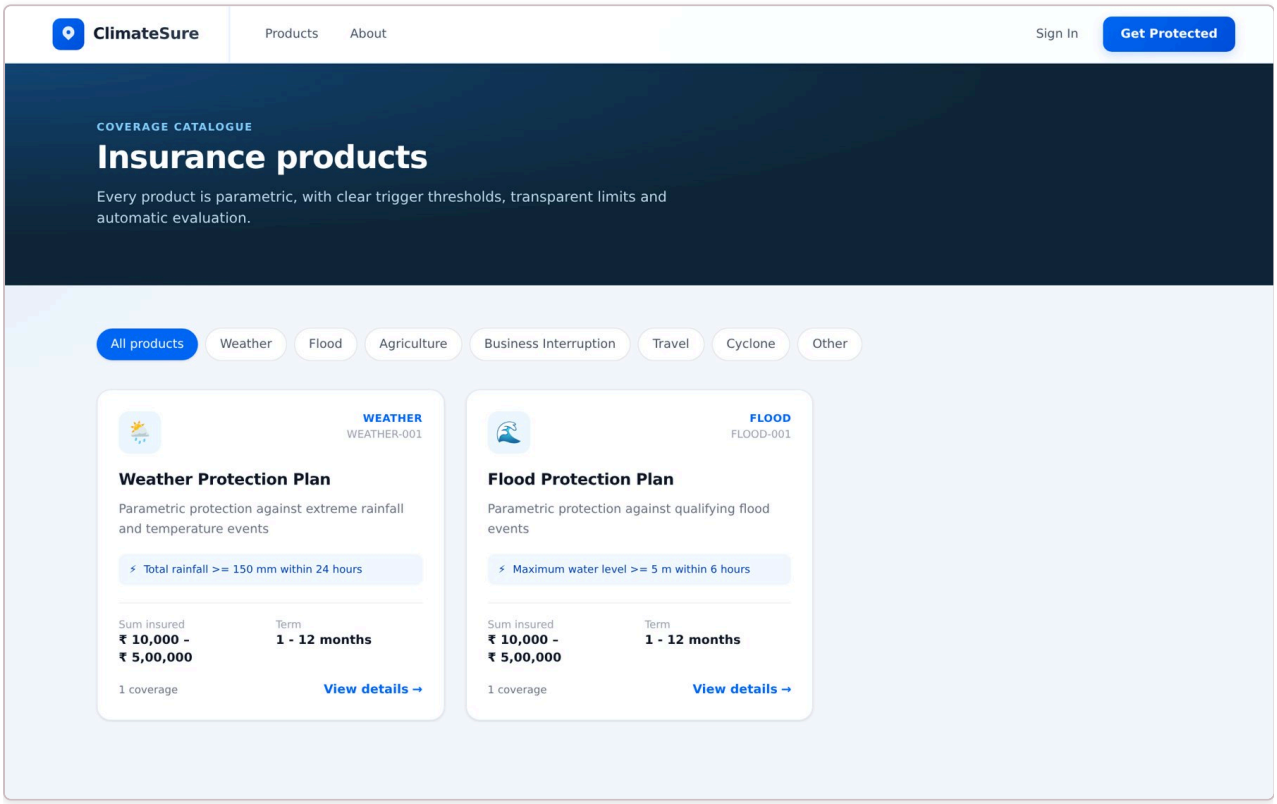


Figure 13.3: Public product catalogue with trigger rules

13.2 Customer Workspace (akshaya@gmail.com)

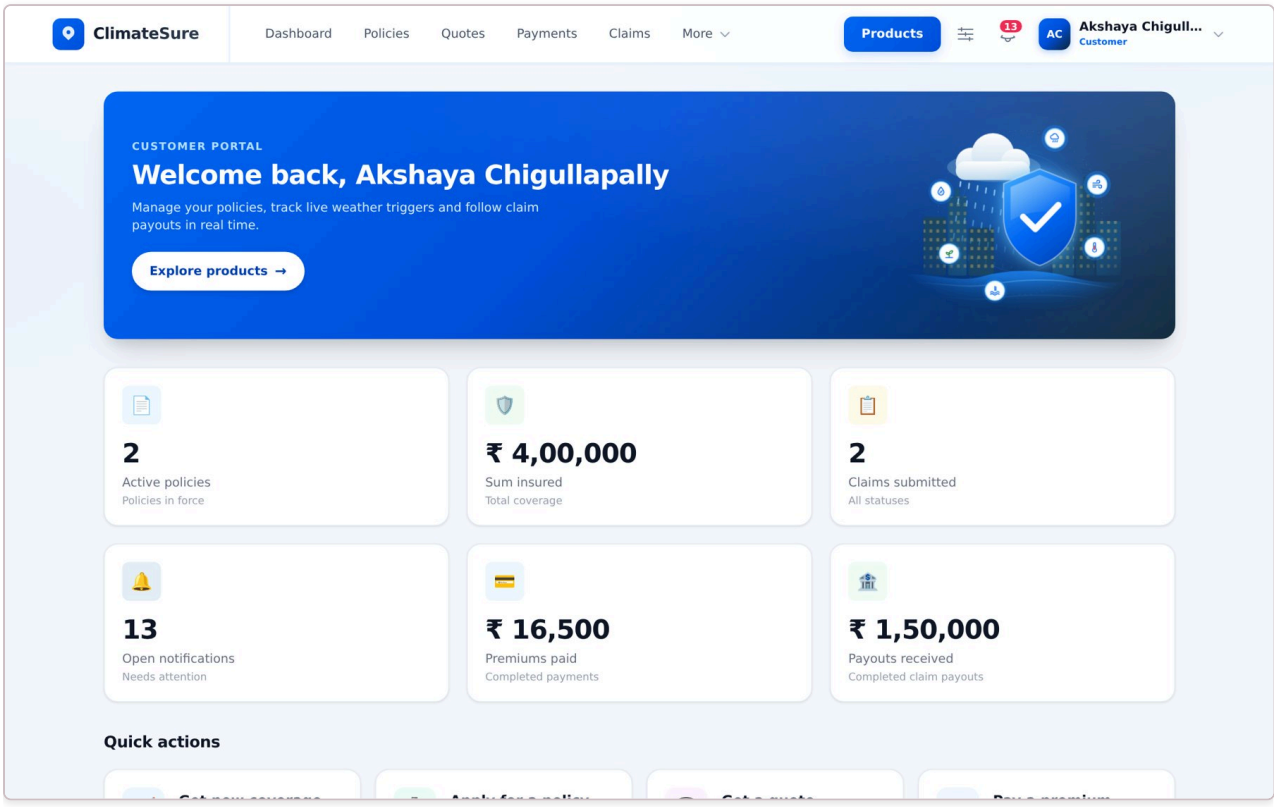


Figure 13.4: Customer dashboard — Akshaya Chigullapally

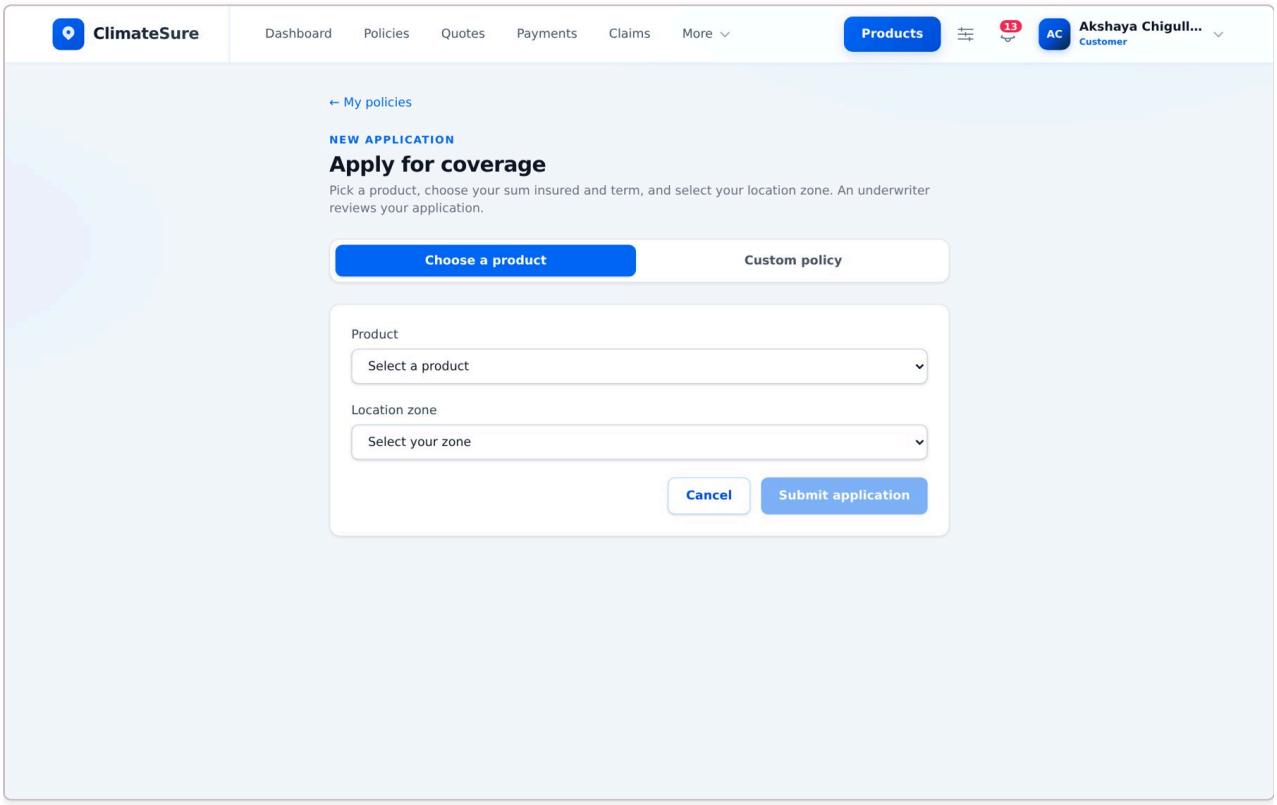


Figure 13.5: Apply for coverage

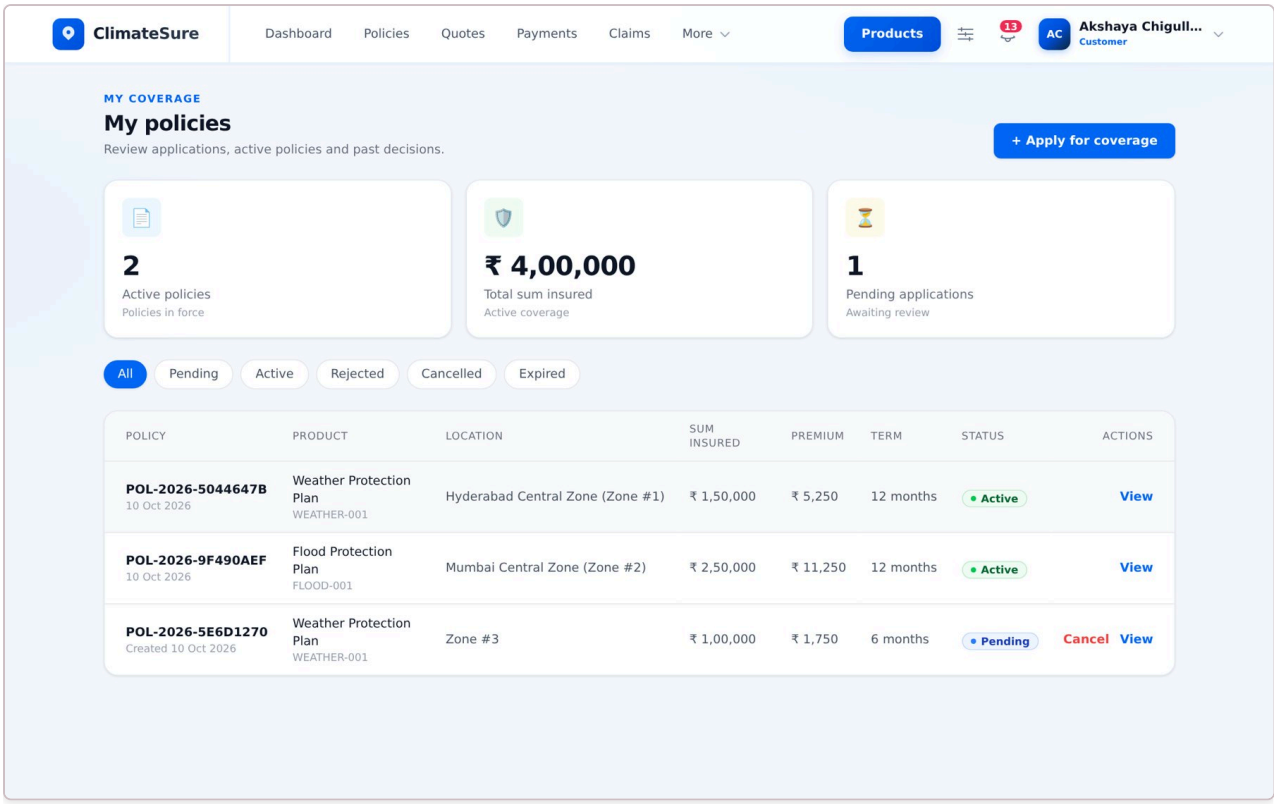


Figure 13.6: My policies — two active policies and one pending application

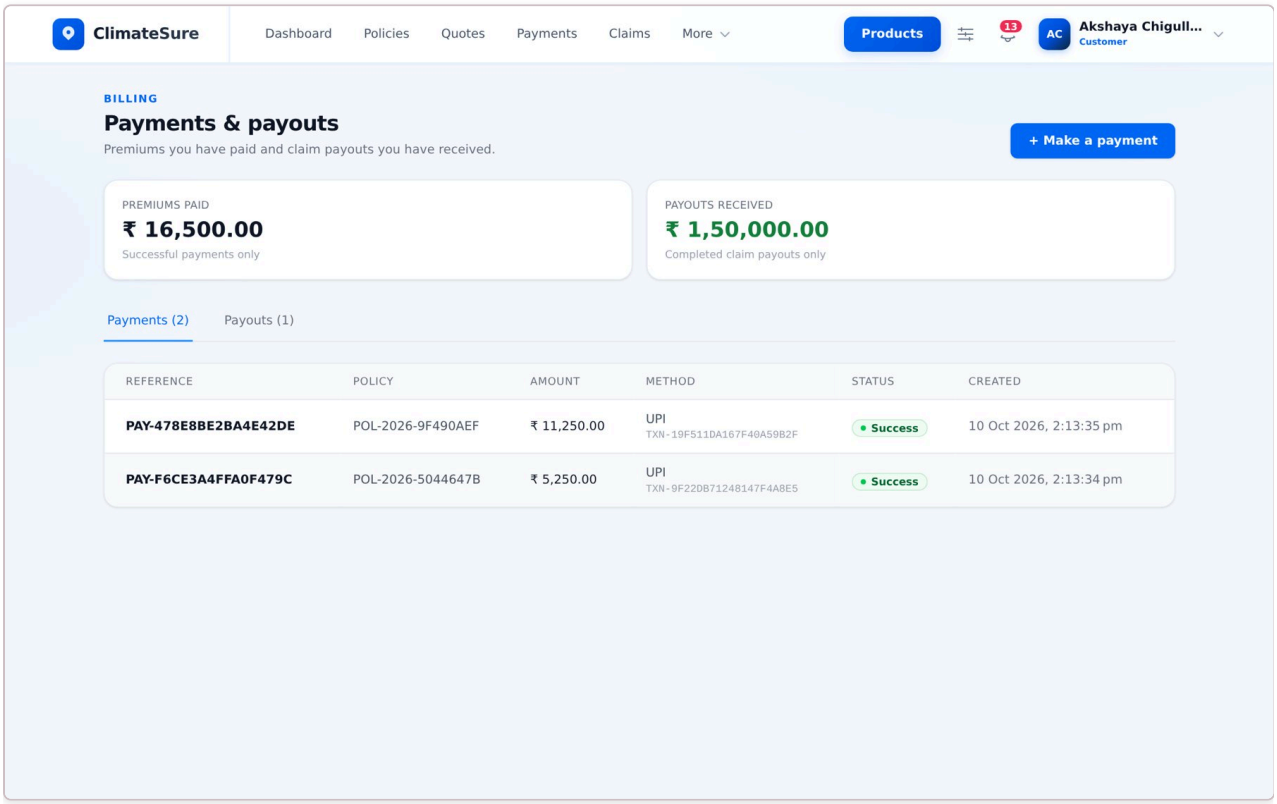


Figure 13.7: Payments and payouts — premiums paid and the ₹1,50,000 payout received

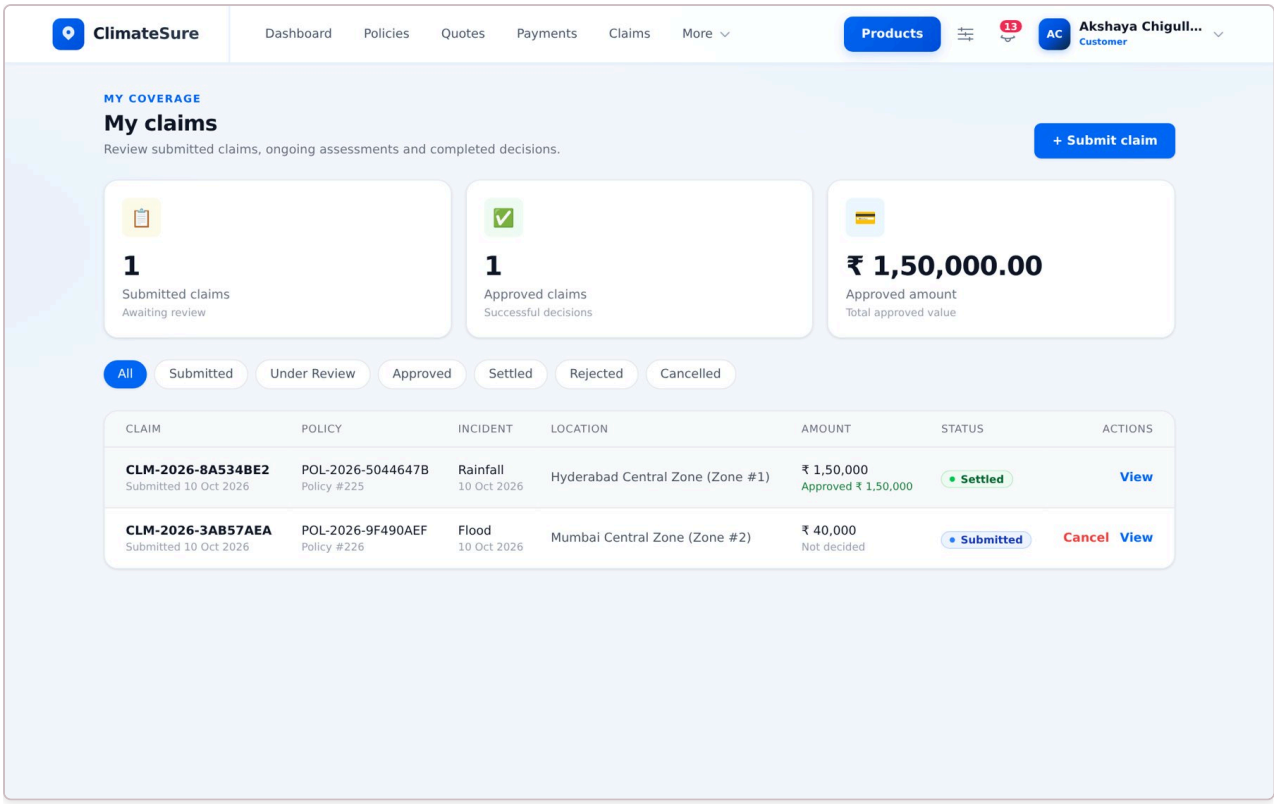


Figure 13.8: My claims — automatic claim settled, manual claim submitted

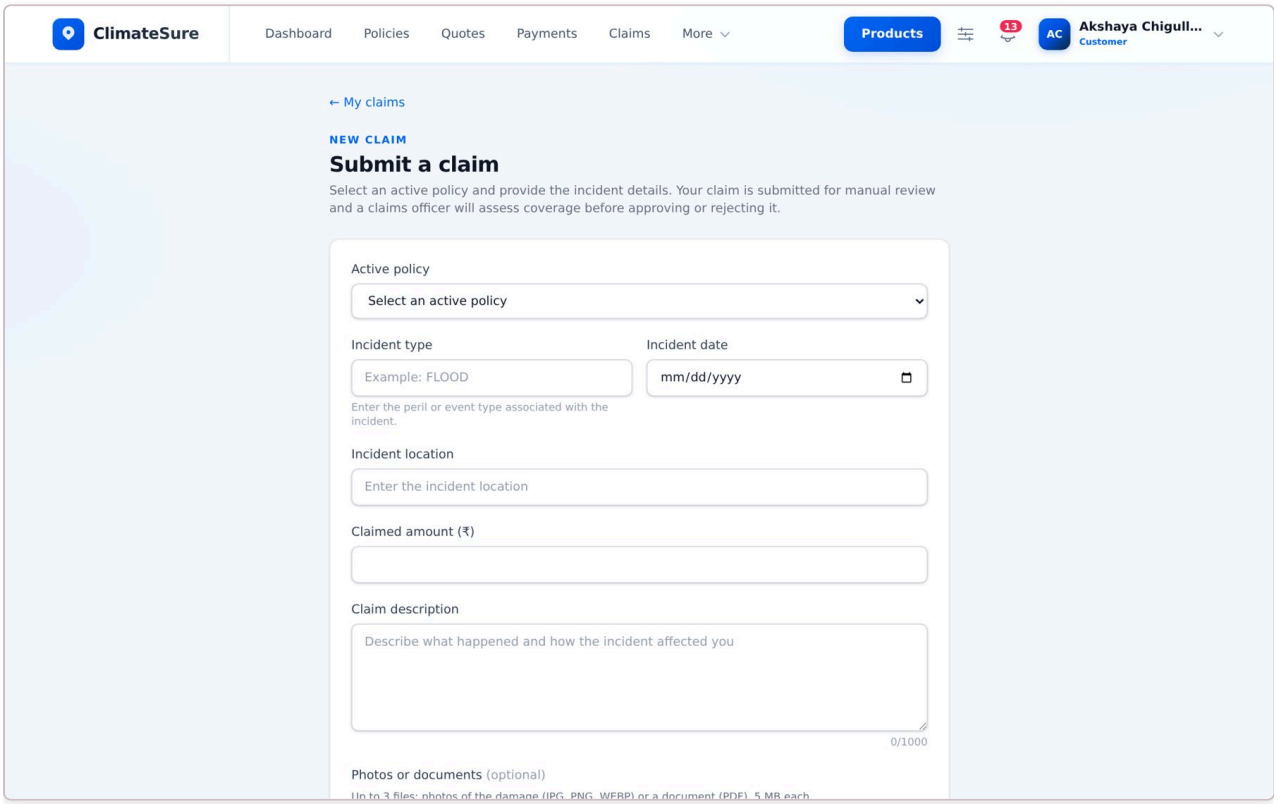


Figure 13.9: Submit a manual claim with evidence

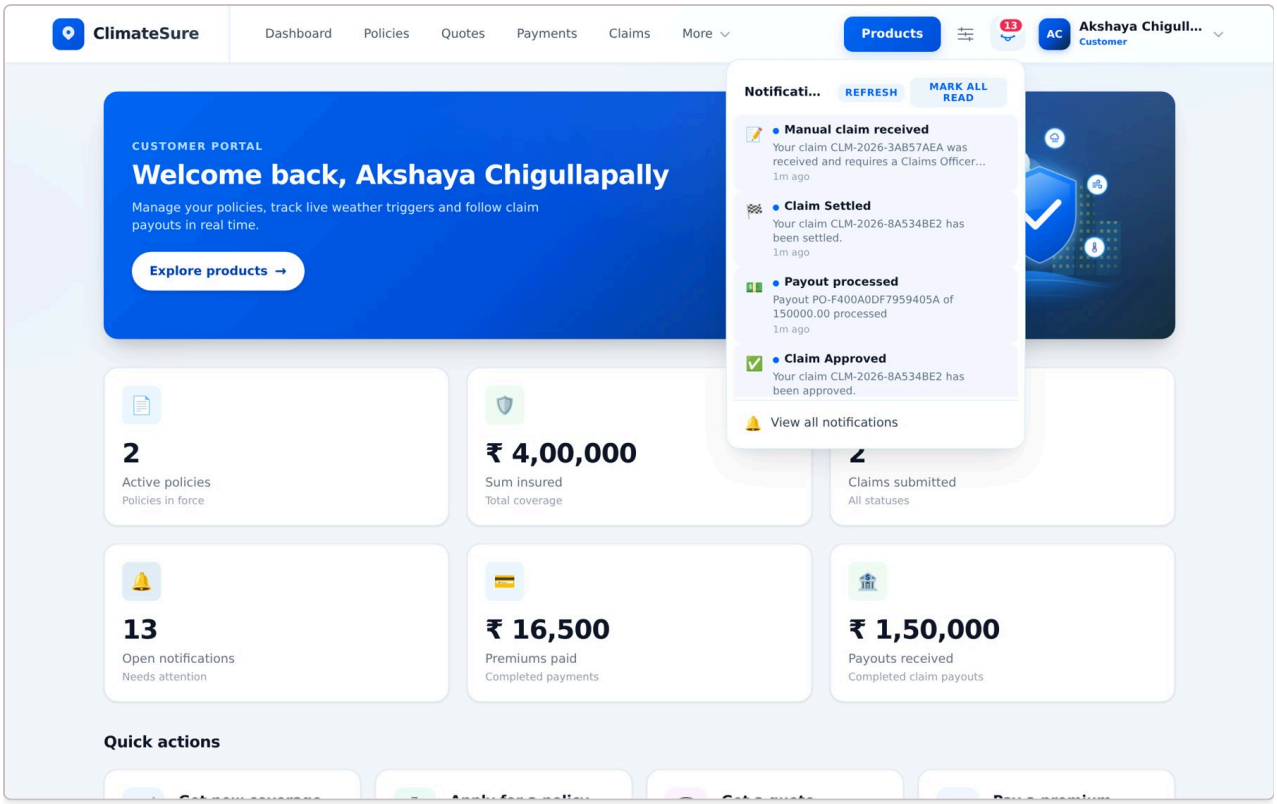


Figure 13.10: Live notifications — claim settled and payout processed

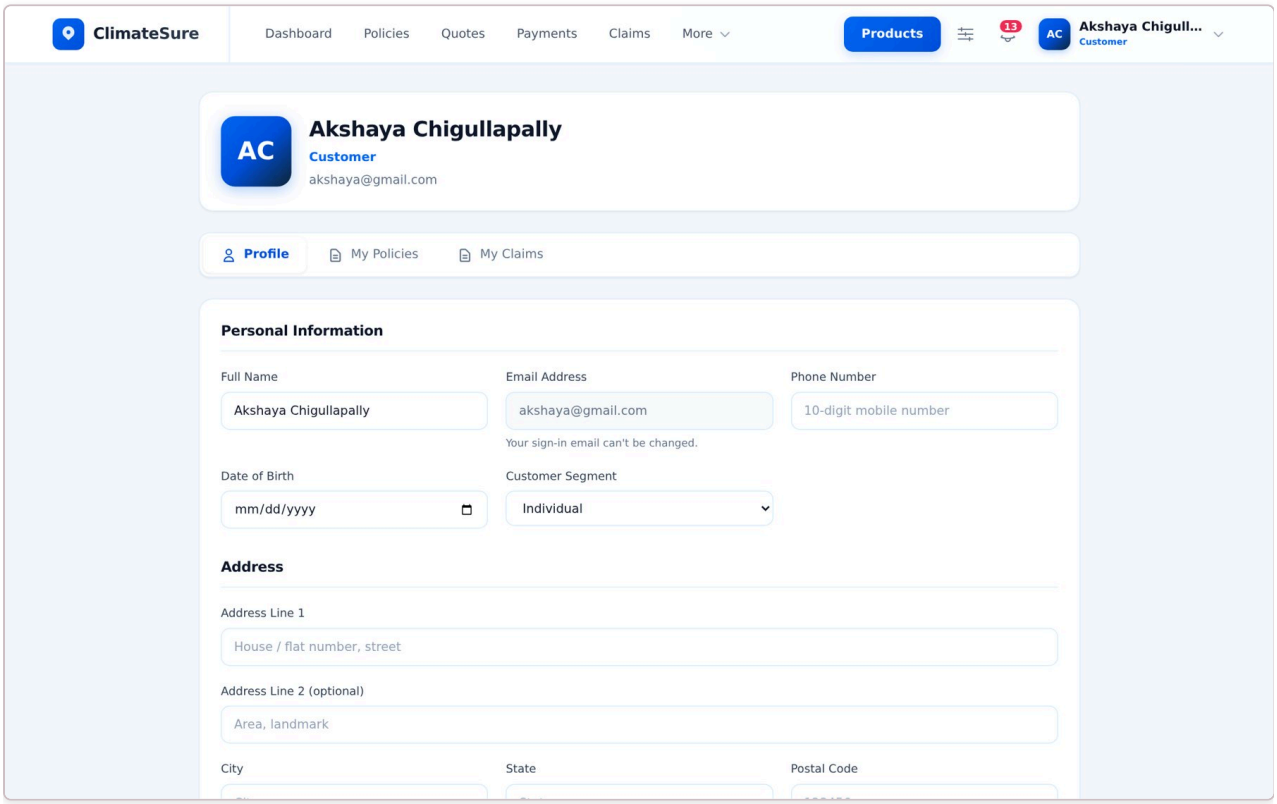


Figure 13.11: Customer profile

13.3 Underwriter Workspace

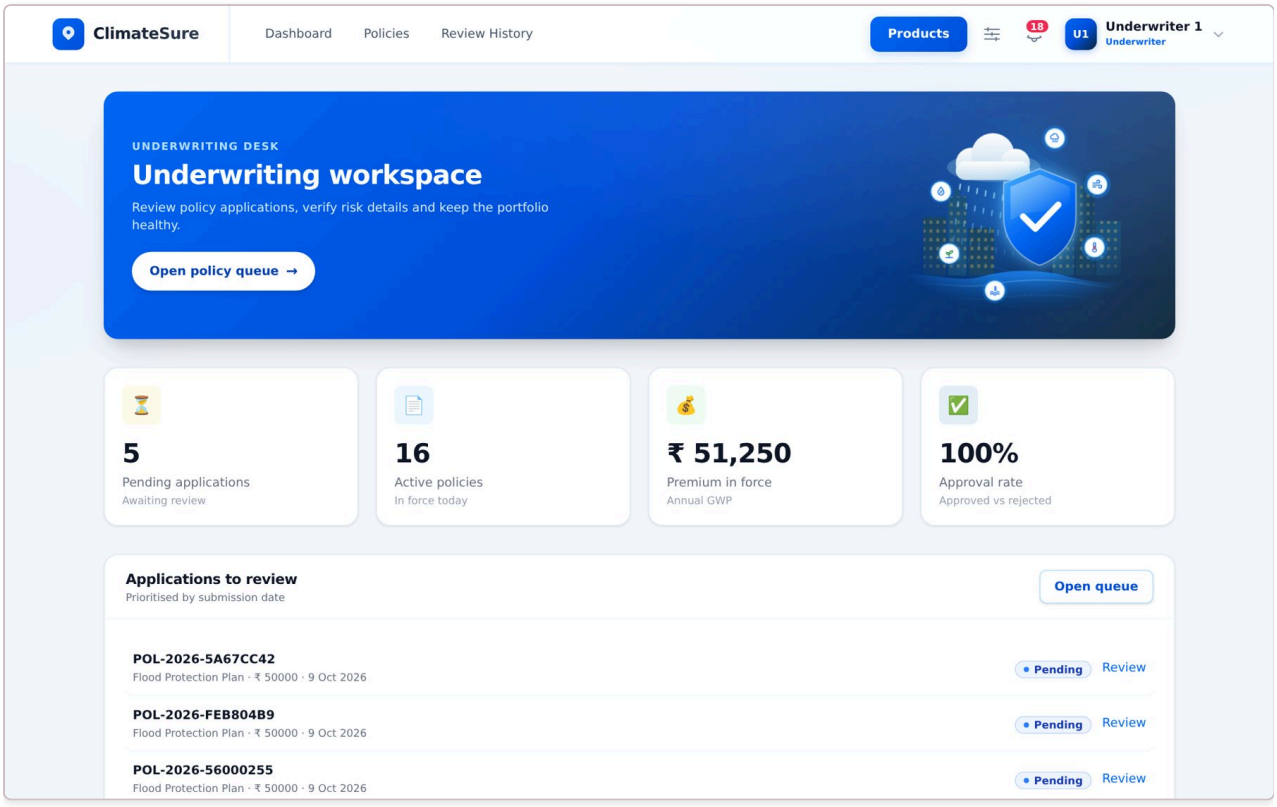


Figure 13.12: Underwriter dashboard

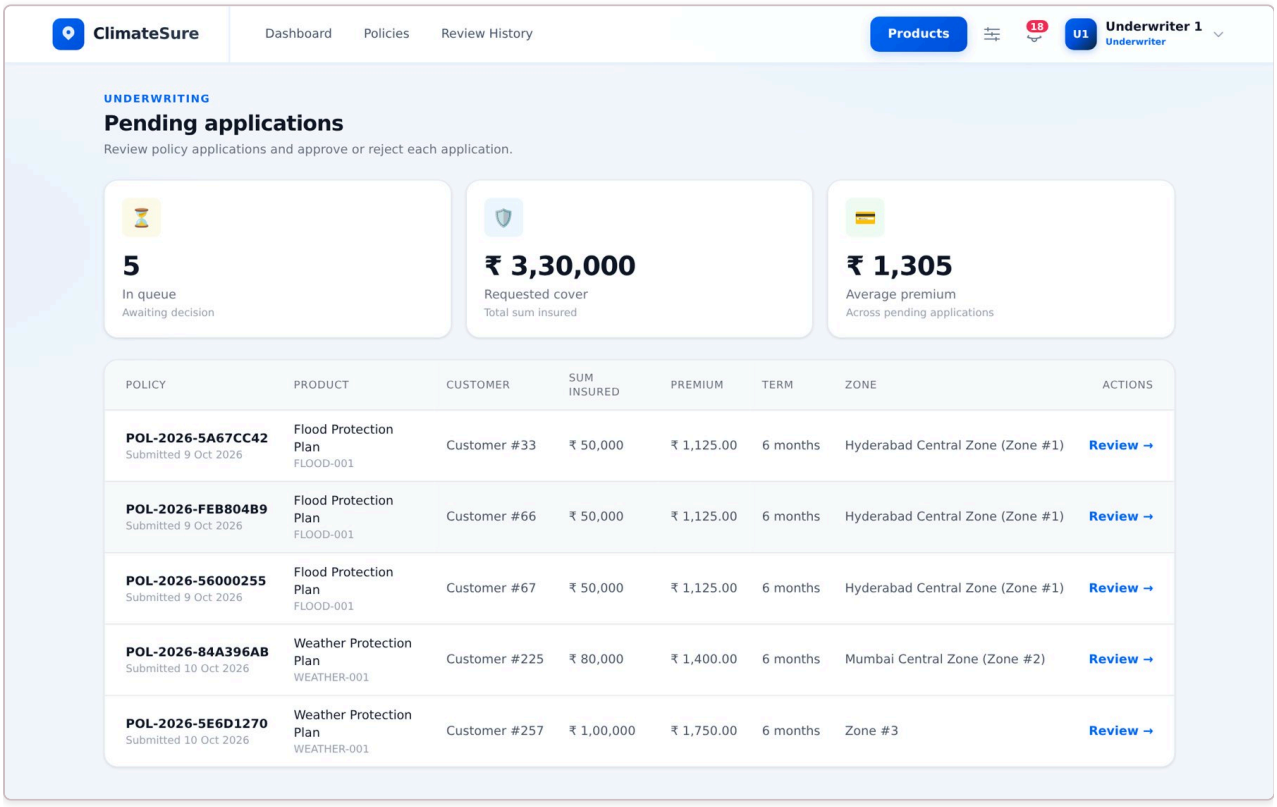


Figure 13.13: Policy applications for review

13.4 Claims Officer Workspace

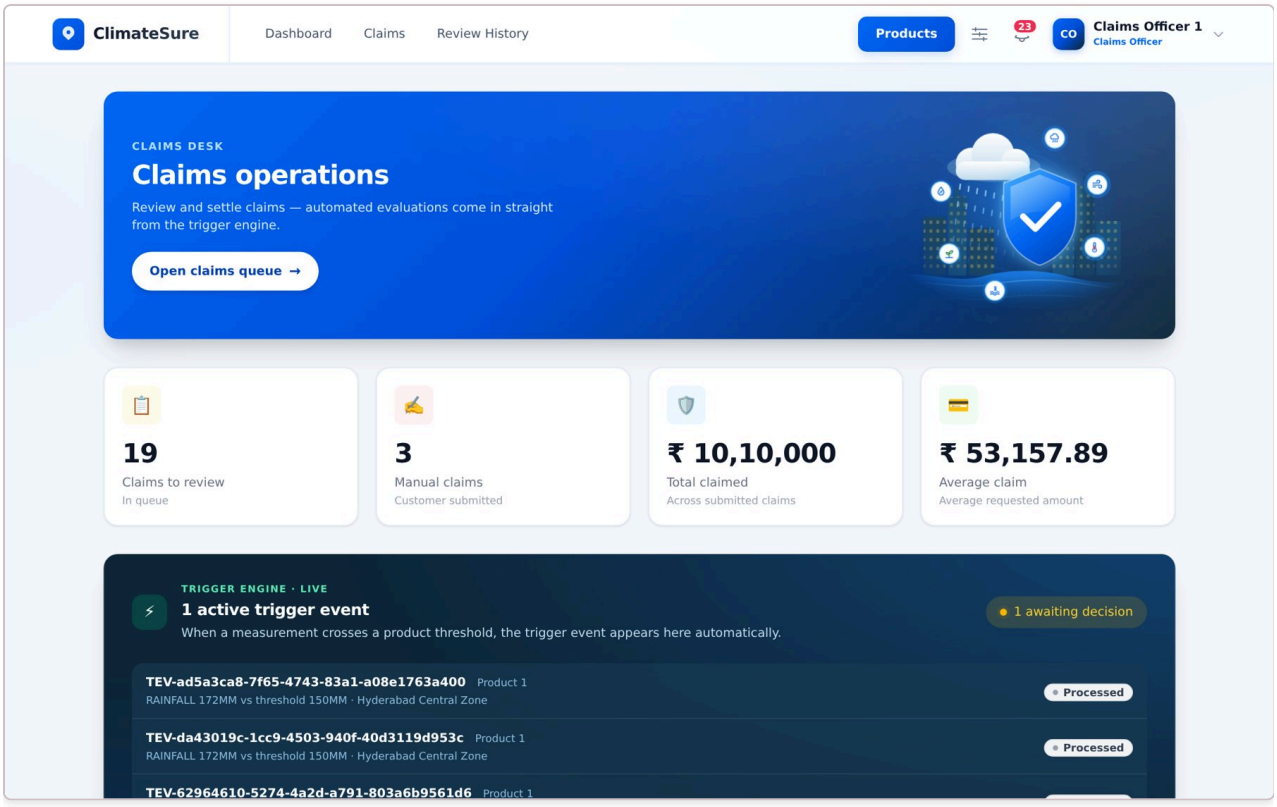


Figure 13.14: Claims officer dashboard

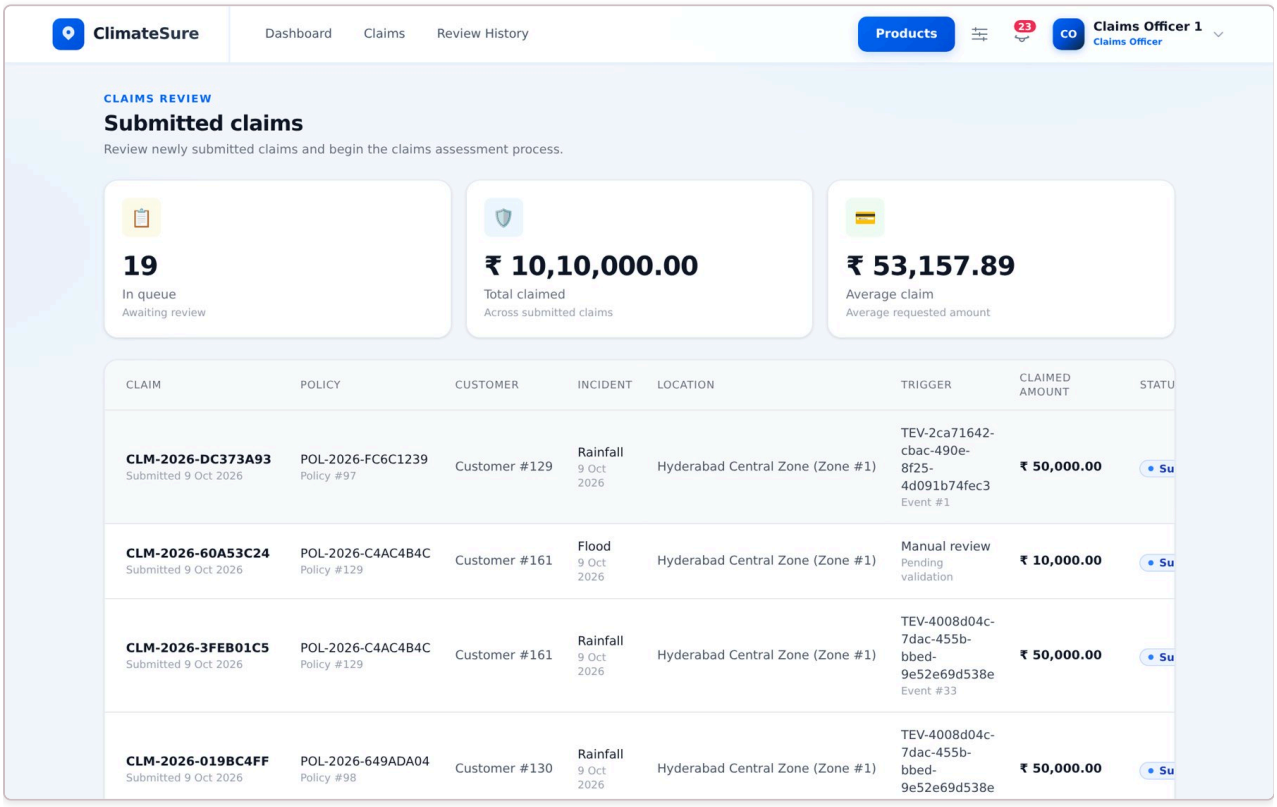


Figure 13.15: Claims review queue

13.5 Admin Workspace

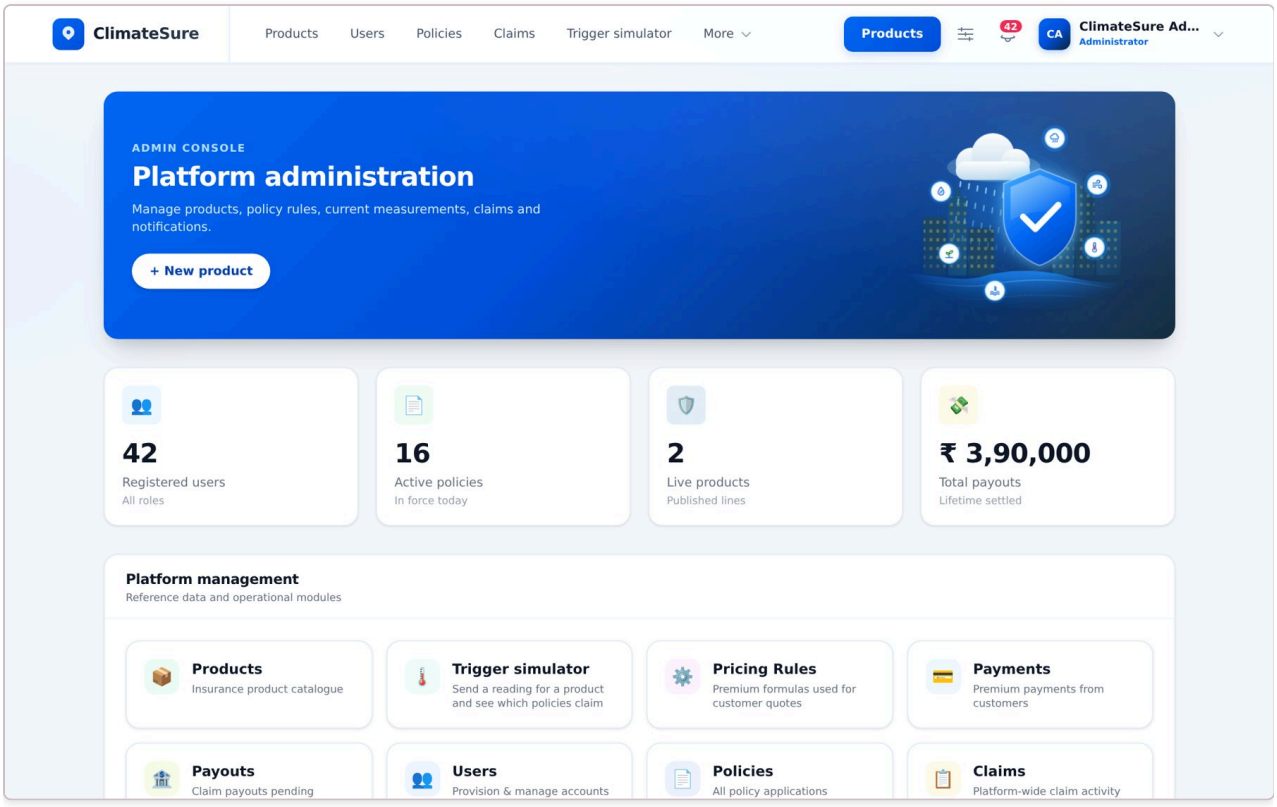


Figure 13.16: Admin dashboard

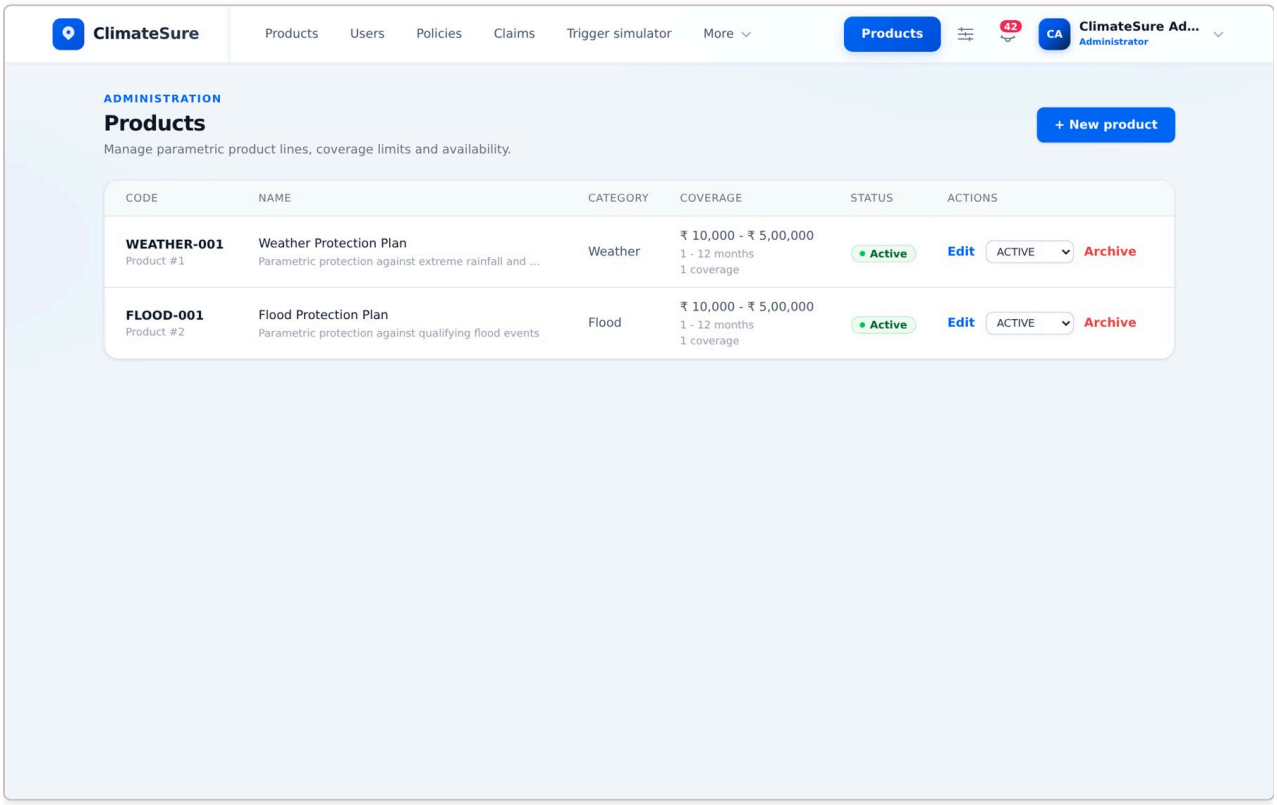


Figure 13.17: Product management

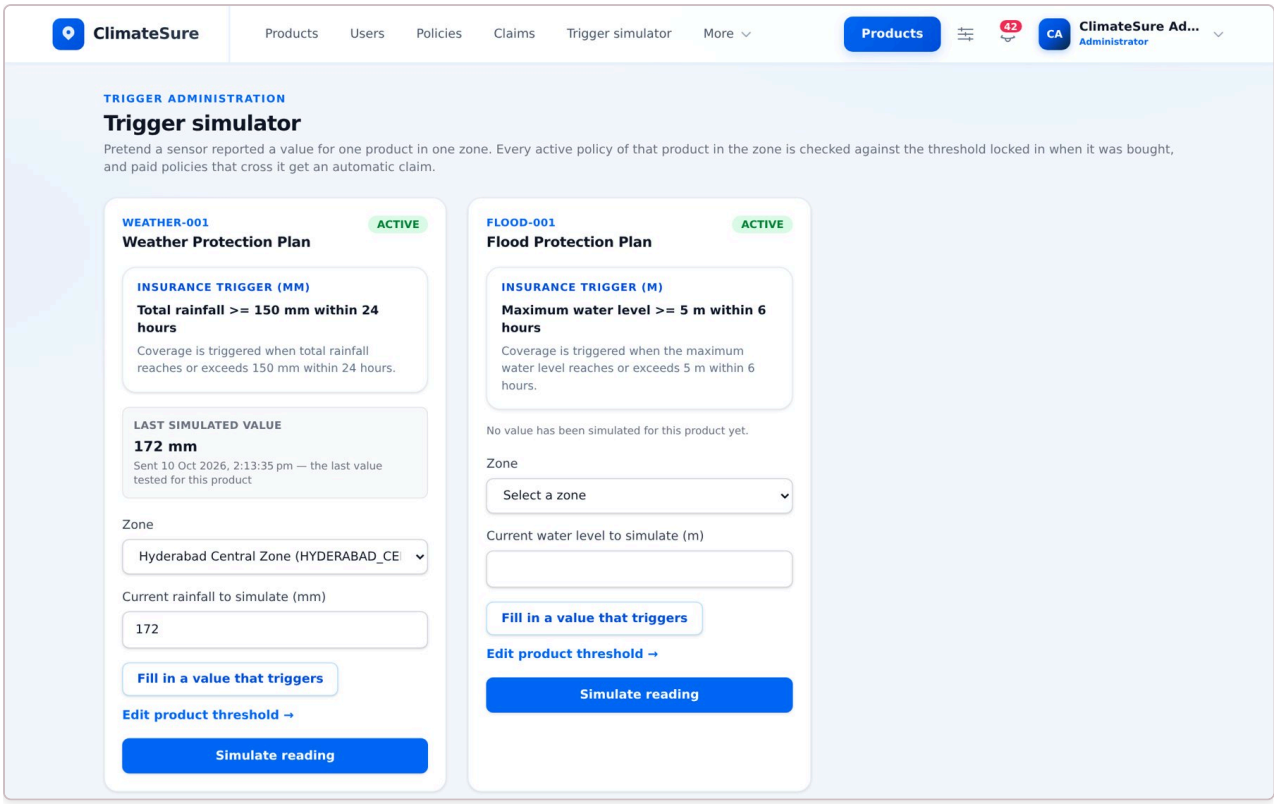


Figure 13.18: Trigger simulator — last simulated value 172 mm for Weather Protection Plan

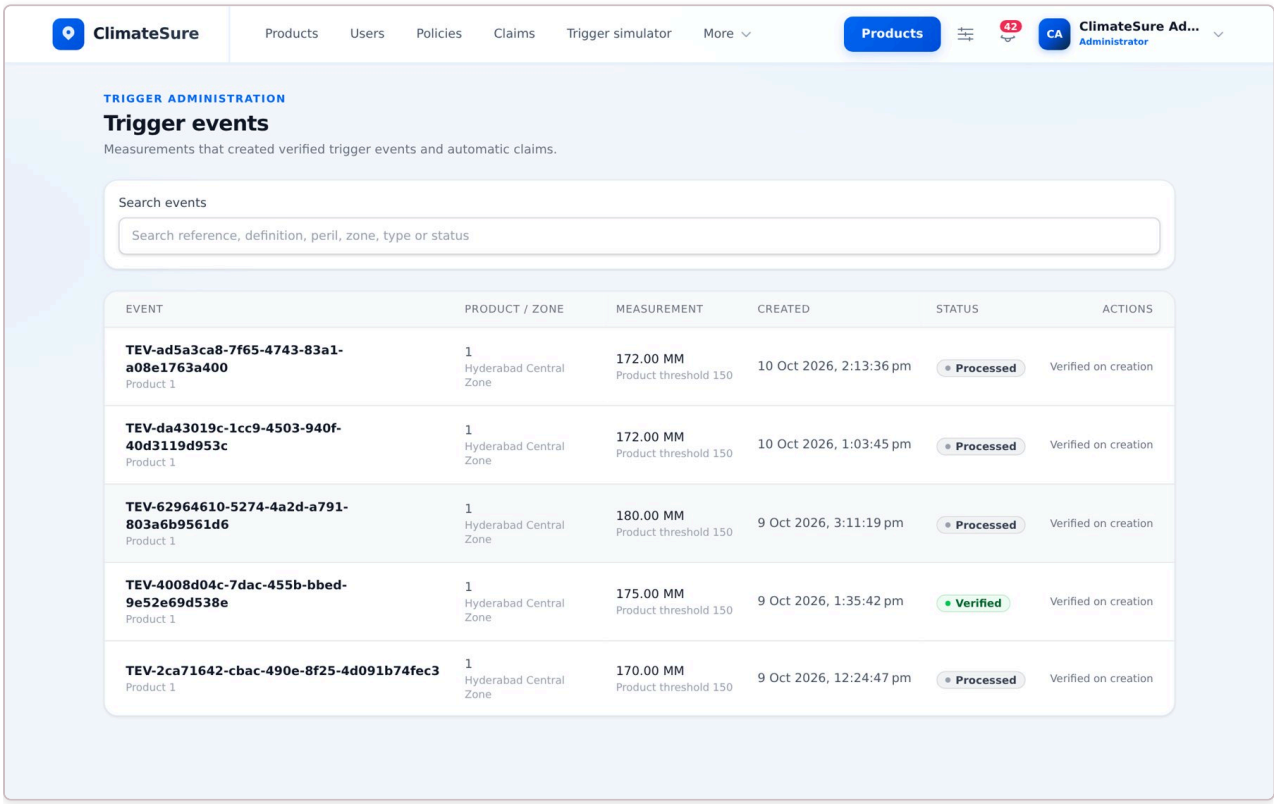


Figure 13.19: Verified trigger events

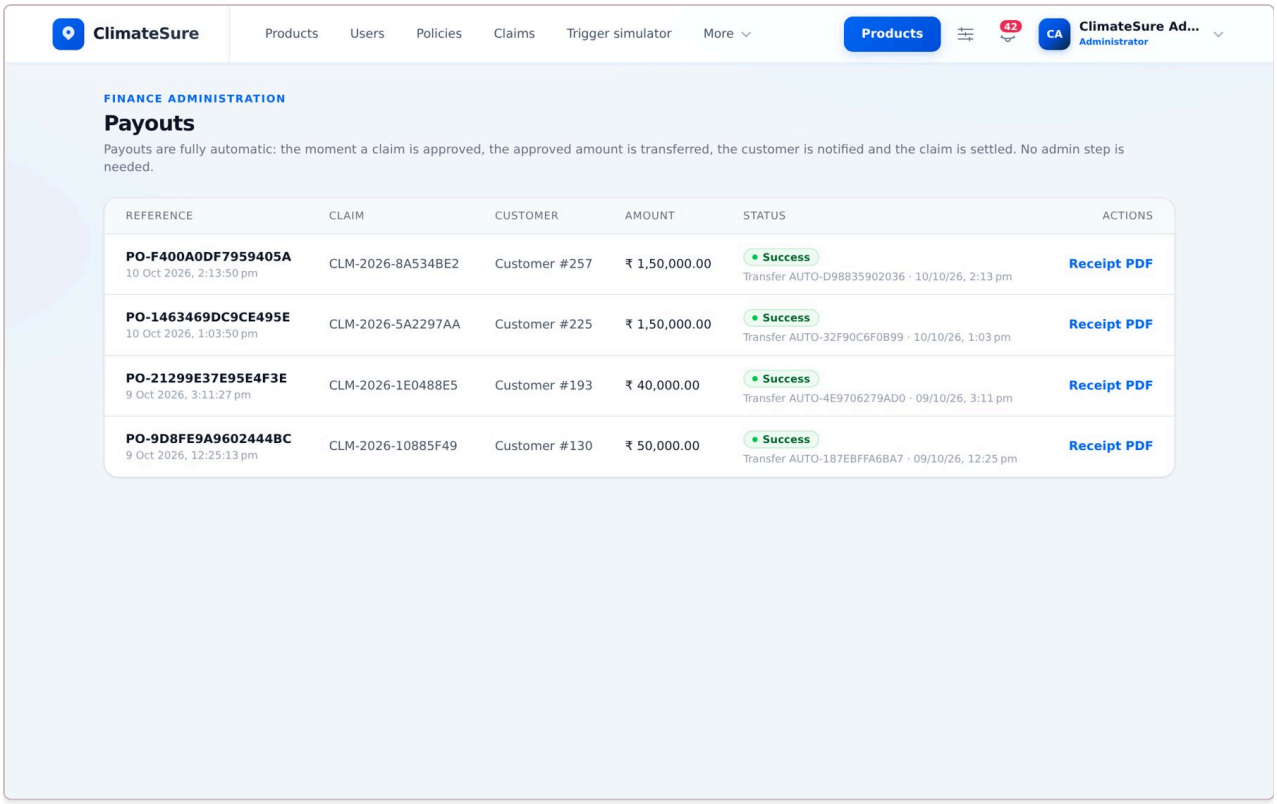


Figure 13.20: Payout history

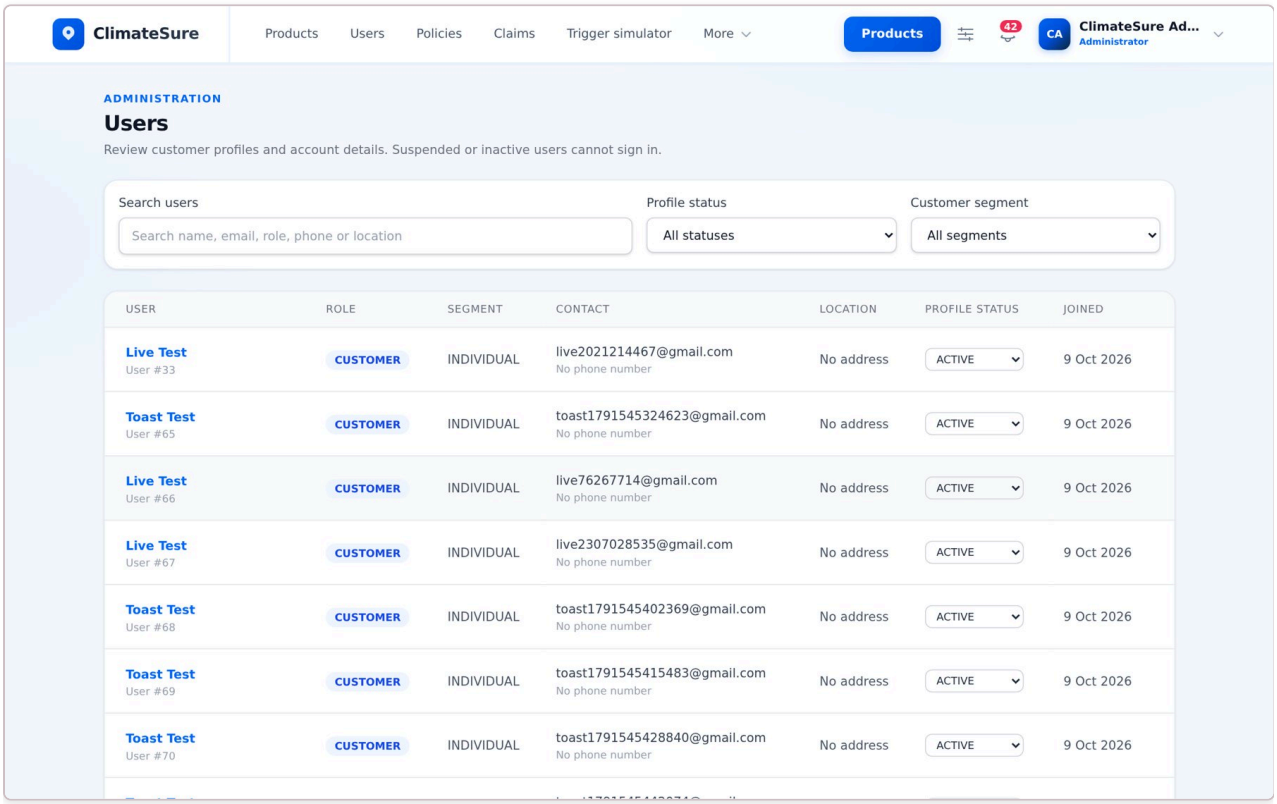


Figure 13.21: User management

13.6 Summary of Results

Table 13.1: Objectives versus results

Objective	Result
Microservices with discovery and gateway	Achieved — 6 services plus gateway and registry, all registered with Eureka
Secure, role-based access	Achieved — JWT with refresh, gateway and service checks, four separate role workspaces
Automatic parametric claims	Achieved — a 172 mm reading created the claim with no customer action
Payout and settlement	Achieved — payout paid immediately on approval, claim settled, event marked processed
Real-time notifications	Achieved — notifications appeared in the bell without refreshing (SSE)
Quality	Achieved — 577 backend and 293 frontend tests passing

CHAPTER 14

14. Assumptions, Limitations & Future Enhancements

14.1 Assumptions

- Measurements entered by the administrator or the demo feed are accurate and come from a trusted source for the zone.
- A location zone is a circle around a central point. A policy covers exactly one zone.
- A single reading of the current value represents the aggregated value for the product's observation window.
- Premium payment and payout disbursement succeed immediately (simulated gateways). Special test card numbers produce declines.
- All amounts are in Indian Rupees. The system runs in a single time zone.
- Staff accounts (admin, underwriters, claims officers) are created by the system. Only customers self-register.

14.2 Limitations

- **No live weather data**: readings come from a demo feed and the admin simulator instead of IMD or a commercial weather API.
- **Simulated payments**: no real payment gateway (Razorpay, Stripe) or bank payout API is integrated.
- **H2 file databases**: suitable for development and demonstration, not for production load or high availability.
- **Synchronous inter-service calls**: a failure in a downstream service is reported as 503. There is no message broker, so the outbox pattern and replay are not available.
- **No distributed tracing**: logs are per service, without a shared correlation id or a central log store.
- **Basis risk**: as in every parametric product, the payout follows the index, not the actual loss. Manual exception claims partly cover this.
- **Single instance**: the services have not been containerised or load-tested in a cluster.

14.3 Future Enhancements

Table 14.1: Planned enhancements

Area	Enhancement
Data	Integrate live weather and river-level APIs (IMD, OpenWeather, Central Water Commission) and satellite indices.
Payments	Integrate a real payment gateway and UPI / bank payouts with webhooks.

Area	Enhancement
Messaging	Introduce Kafka or RabbitMQ for trigger events, claims and notifications (event-driven saga with the outbox pattern).
Resilience	Add Resilience4j circuit breakers, retries and rate limiting at the gateway.
Database	Move to PostgreSQL with Flyway migrations, and add read replicas for reporting.
Observability	Micrometer tracing with Zipkin, Prometheus and Grafana dashboards, and ELK for central logs.
Deployment	Docker images, Docker Compose / Kubernetes, CI/CD with GitHub Actions and a central config server.
Security	OAuth2 / OpenID Connect, multi-factor authentication, refresh-token rotation and HTTP-only cookies.
Product	Multi-zone and multi-peril policies, tiered (partial) payouts, and a mobile app with push notifications.
Analytics	Risk scoring and dynamic pricing from historical weather data, and fraud detection on manual claims.

CHAPTER 15

15. Conclusion

ClimateSure shows that parametric insurance can be delivered end to end on a modern, reactive microservice platform. Measurable trigger rules, locked into each policy at purchase, let the system move from a weather reading to a settled claim and a payout in seconds, with a human officer still in control of the final approval. The microservice design keeps each business capability independent: authentication, products and policies, claims, triggers, finance and notifications each have their own database and can evolve and scale separately. The gateway and the shared security model protect them consistently.

The project combines Spring WebFlux, R2DBC, Spring Cloud Gateway, Eureka, Spring Security with JWT, and an Angular and NgRx frontend with live Server-Sent Events. It is supported by input validation on DTOs, uniform error handling, structured logging, actuator monitoring and 870 automated tests. Limitations remain around live data sources, real payment gateways and production infrastructure. The architecture is ready for these extensions, and the roadmap in Chapter 14 sets out how the platform can grow into a production-grade insurance system.

References

1. Spring Boot Reference Documentation, version 4.x. https://docs.spring.io/spring-boot/
2. Spring Framework — Web on Reactive Stack (WebFlux). https://docs.spring.io/spring-framework/reference/web/webflux.html
3. Spring Data R2DBC Reference. https://docs.spring.io/spring-data/relational/reference/r2dbc.html
4. Spring Cloud Gateway and Spring Cloud Netflix (Eureka) Reference. https://spring.io/projects/spring-cloud
5. Spring Security Reference — Reactive Applications. https://docs.spring.io/spring-security/reference/reactive/
6. M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015.
7. Jakarta Bean Validation 3.0 Specification. https://jakarta.ee/specifications/bean-validation/
8. Angular Documentation. https://angular.dev/
9. NgRx Documentation. https://ngrx.io/
10. Vitest Documentation. https://vitest.dev/ · JaCoCo Java Code Coverage Library. https://www.jacoco.org/
11. C. Richardson, *Microservices Patterns*, Manning Publications, 2018.
12. World Bank / Global Index Insurance Facility, "Index-based (parametric) insurance — an overview".

Appendix A — Running the Project

```
# 1. Backend (Java 17 + Maven), in this order
cd ParametricInsurance/EurekaServer       && mvn spring-boot:run   # :8761
cd ParametricInsurance/authservice        && mvn spring-boot:run   # :8081
cd ParametricInsurance/productservice     && mvn spring-boot:run   # :8082
cd ParametricInsurance/claimsservice      && mvn spring-boot:run   # :8083
cd ParametricInsurance/notificationservice&& mvn spring-boot:run   # :8085
cd ParametricInsurance/triggerservice     && mvn spring-boot:run   # :8087
cd ParametricInsurance/financialservice   && mvn spring-boot:run   # :8088
cd ParametricInsurance/apigateway         && mvn spring-boot:run   # :8080

# 2. Frontend (Node 20+)
cd InsuranceFrontend && npm install && npm start                    # http://localhost:4200

# 3. Tests
mvn test                        # in each service (JaCoCo report in target/site/jacoco)
npx ng test --watch=false --coverage
```

Table A.1: Accounts used for the results

Role	Account used in this report
Customer	akshaya@gmail.com (Akshaya Chigullapally)
Admin	admin@gmail.com
Underwriter	uw1@gmail.com
Claims officer	co1@gmail.com